

DFIR at Machine Speed

One Rust-native toolchain, from raw image to board-ready narrative

BSidesHK 2026 · Blue-Team Workshop · 3 hours, hands-on

Albert Hui — Security Ronin · IR Peer Review: Eliza Wan · TA: Josiah Wu

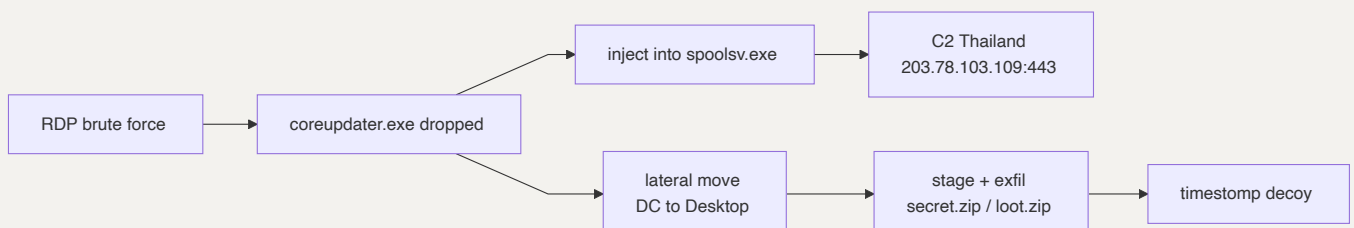
Case 001 — "The Stolen Szechuan Sauce" · disk + RAM only · two real Windows hosts

The Scenario

A Windows estate breached on **19 September 2020**:

- Attacker **brute-forces RDP** into a Domain Controller
- Drops **Meterpreter** / `coreupdater.exe`, injects into `spoolsv.exe`
- Beacons to a **C2 in Thailand** (`203.78.103.109:443`)
- Moves laterally **DC → Win10 desktop**, stages and **exfiltrates secrets**
- **Time-stomps a decoy** — and is *still interactive* at the moment of capture

You are the IR analyst. You receive the evidence cold. **Build the story.**

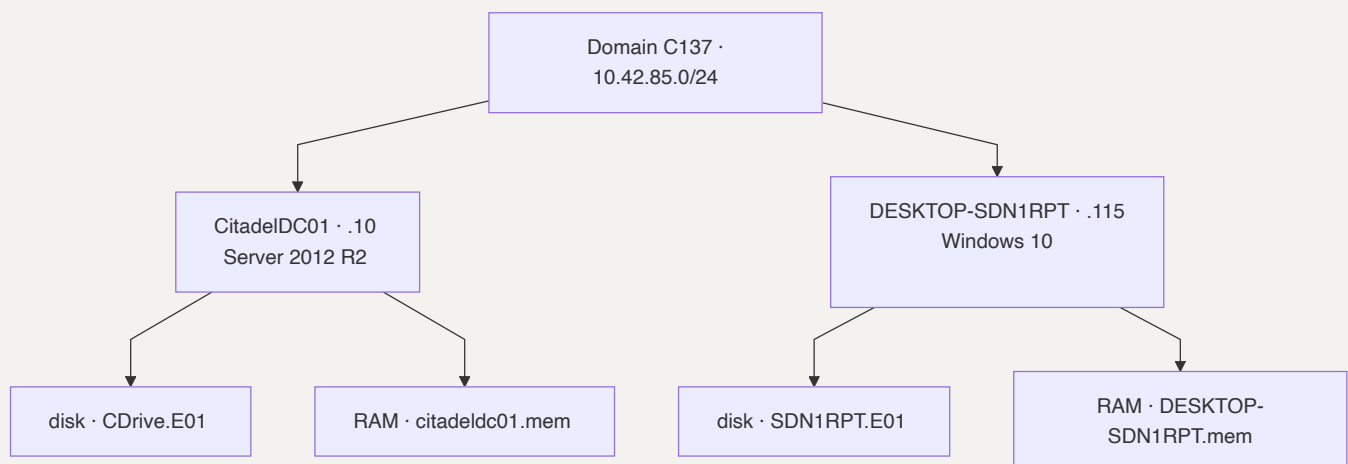


The Evidence You Receive

Two victim hosts on domain **C137** (10.42.85.0/24):

HOST	ROLE	OS	DISK IMAGE	MEMORY
CitadelDC01 .10	Domain Controller	Server 2012 R2	... CDrive.E01	citadeldc01.mem
DESKTOP-SDN1RPT .115	Workstation	Win 10 Enterprise	... SDN1RPT.E01	DESKTOP-SDN1RPT.mem

≈ **12.8 GB** total. Pre-staged on your USB stick / download link.



The Full Case 001 Artifact Set

Everything DFIR Madness publishes for this case (<https://dfirmadness.com/case001/>):

Domain Controller (CitadelDC01)

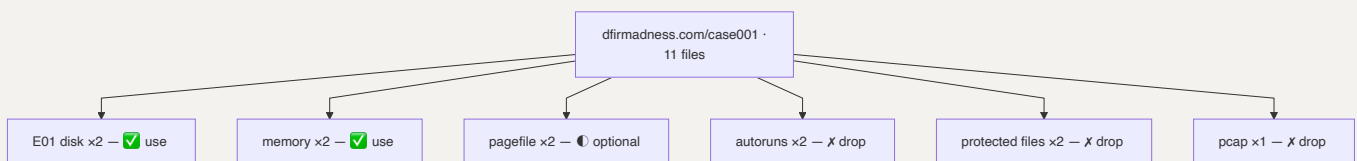
- DC01-E01.zip — disk image · DC01-memory.zip — RAM · DC01-pagefile.zip
- DC01-autorunsc.zip · DC01-ProtectedFiles.zip

Workstation (DESKTOP-SDN1RPT)

- `DESKTOP-E01.zip` · `DESKTOP-SDN1RPT-memory.zip` · `Desktop-SDN1RPT-pagefile.zip`
- `DESKTOP-SDN1RPT-autorunsc.zip` · `DESKTOP-SDN1RPT-Protected Files.zip`

Network

- `case001-pcap.zip`



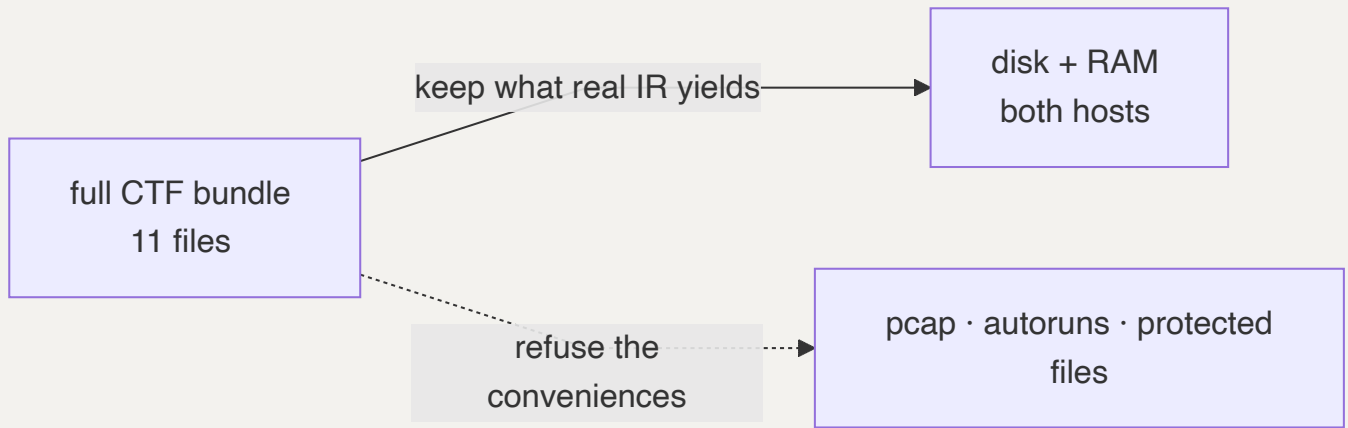
What We Use Today — and Why

☒ **In scope: disk image + RAM dump** for *both* hosts. Nothing else.

This is **not** us simplifying the case. It is us **mimicking real post-incident IR**:

- In a real engagement you almost always get a **dead disk and (if you're lucky) a memory capture** — pulled after the fact.
- Everything else on that download page is a **convenience the CTF pre-cooked for you**. We refuse the convenience on purpose.

The skill we are training is working from what you actually get, not from a tidy artifact bundle.

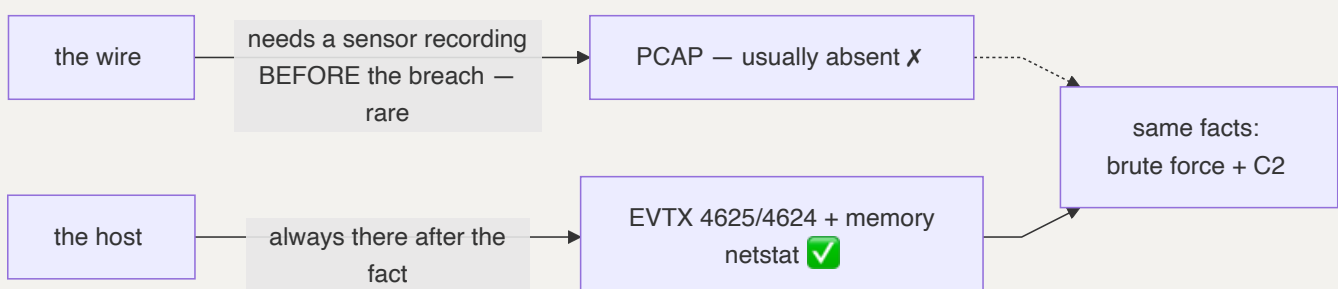


Why No PCAP

`case001-pcap.zip` is **excluded** — deliberately.

- Full packet capture means **someone was already recording the wire** before/at the breach. In the field that is **rare** — most orgs have no retained PCAP at the moment that matters.
- Relying on PCAP teaches a habit that **breaks the day you don't have it**.
- The *outcomes* PCAP would show — the brute force, the C2 — are **independently provable** from disk (EVTX 4625/4624) and memory (netstat). We reconstruct them from artifacts that **survive**.

PCAP-only details (an NMAP 3389 probe at 02:19) become a **footnote**, not an assessable question.

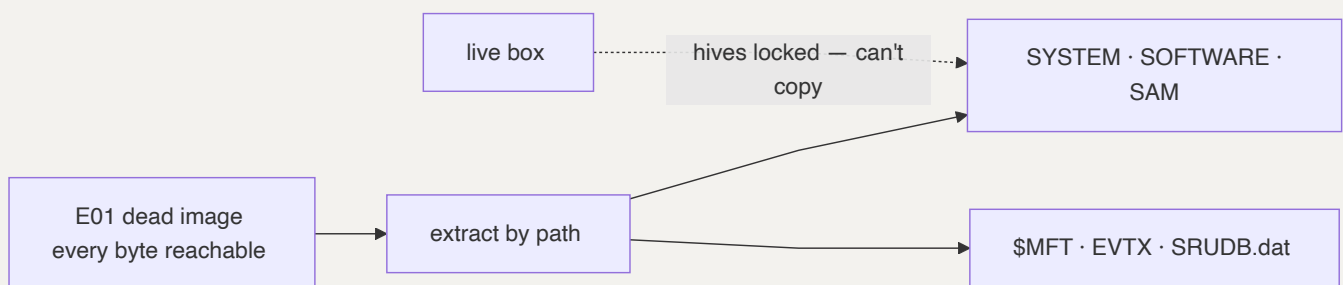


Why We Extract the System Files Ourselves

`*-autorunsc.zip` and `*-ProtectedFiles.zip` are **excluded** — also deliberately.

- Those are **pre-extracted hives, autoruns, locked files** — work a tool already did *in the lab*.
- Pulling `SYSTEM` / `SOFTWARE` / `SAM`, `$MFT`, `EVTX`, `SRUDB.dat` **out of the E01 by path** is a **core lab step** — so we do it ourselves, live.
- Locked/"protected" files (loaded hives, `pagefile.sys`) can't be copied off a live box normally — but on a **dead image every byte is reachable**. That's the lesson.

Extraction is the exercise. You leave knowing how the sausage is made.

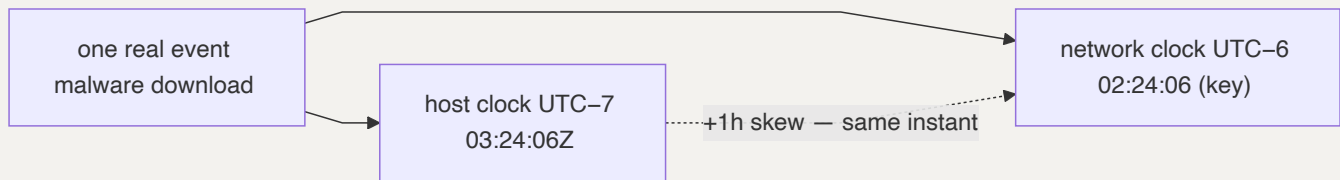


One Trap to Internalize: The Clock

The victim VMs were **mis-configured to UTC-7**. The (excluded) PCAP router was **UTC-6**.

- Disk / EVTX / memory timestamps read **~1 hour ahead** of the network-clock narration in the official key.
- The key's `02:24:06` download = your tooling's `03:24:06Z` — *same instant, different clock*.

Always establish clock provenance before you trust a timeline. Issen surfaces this via `ClockProvenance` so the skew is a labeled fact, not a silent error.



The Real Point of This Workshop

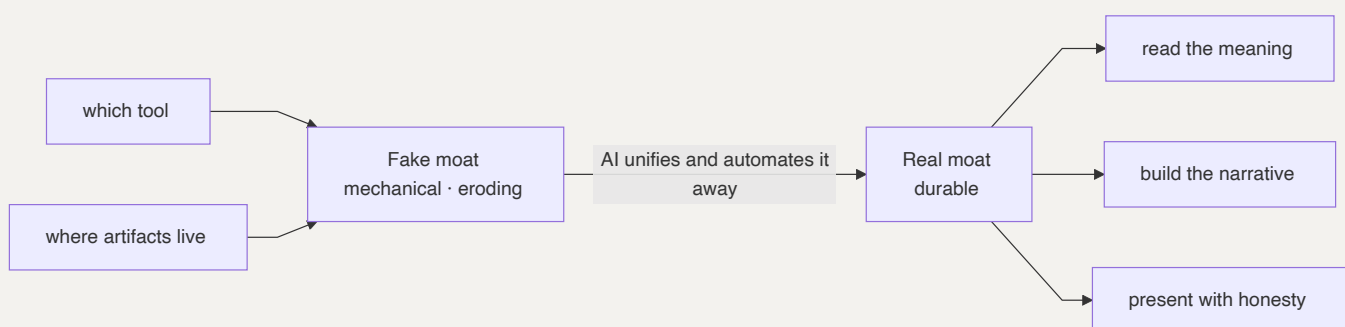
Knowing *which tool* and *where the artifact lives* feels like expertise. It is a **fake moat**:

- It is **mechanical** — lookup-table knowledge.
- In the age of AI it is being **unified, normalized, and automated away**.

The **real moat** is the **investigative mindset**:

- Reading what the output **means**
- Building the **attack narrative**
- **Presenting it** to a board with intellectual honesty

*We spend the mechanical time in **one** tool so the cognitive time goes where it counts.*

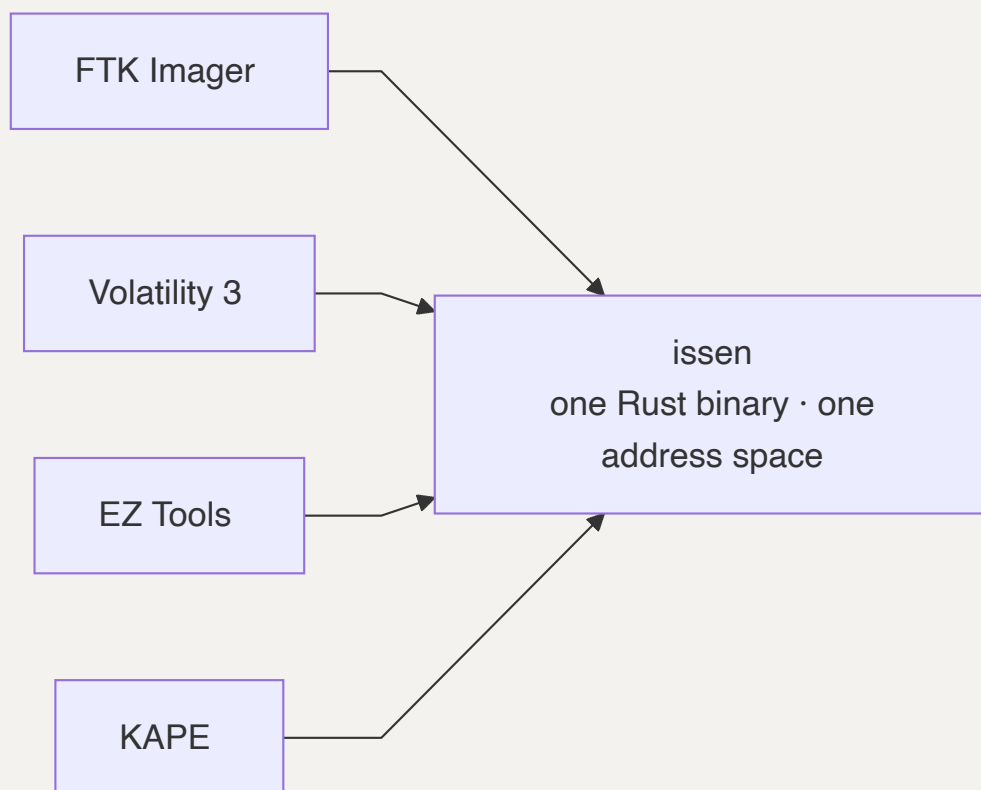


Why Issen Is Different

The traditional path: **FTK Imager** + **Volatility** + **Eric Zimmerman tools** + **KAPE** — four ecosystems, three languages, two OSes, glue scripts in between.

Issen's bet:

- **One cross-platform binary.** Native macOS / Windows / Linux. Rust. `cargo install`, no runtime.
- **One address space for the whole case** — disk, memory, logs converge into a single timeline.
- **Forensically paranoid by construction** — panic-free parsers, never trust a length field, fail loud on the unknown.
- **Findings, not verdicts** — every output is "*consistent with*", leaving the conclusion to you.

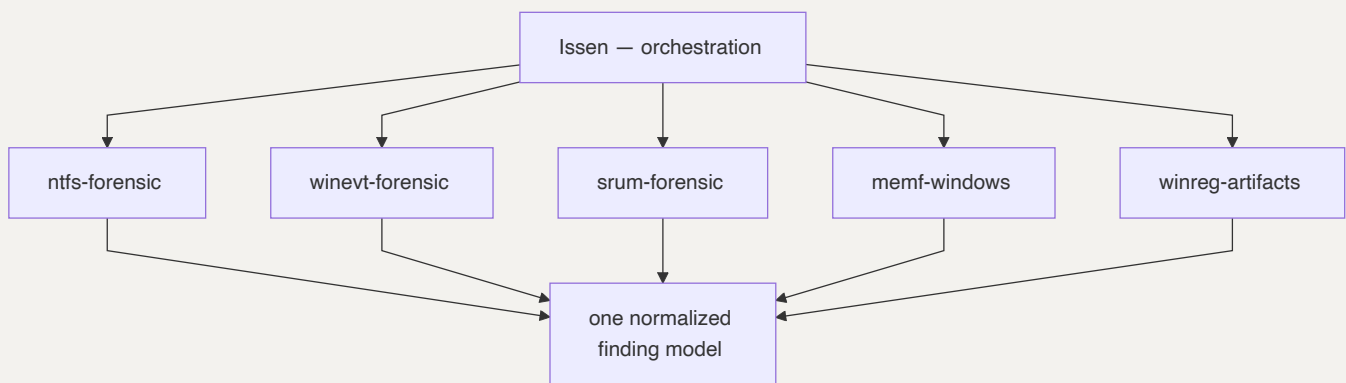


It's Not One Tool — It's a Fleet

Issen is a thin **orchestration layer** over a family of standalone, single-purpose forensic libraries.

- Each library is a **deep expert** in one artifact family (NTFS, EVTX, SRUM, memory paging...).
- Issen **wires them together** and correlates across them.
- Every library emits the **same normalized finding model**, so one report renders them uniformly.

The architecture is organized around **how an analyst navigates evidence** — five fundamental primitives.

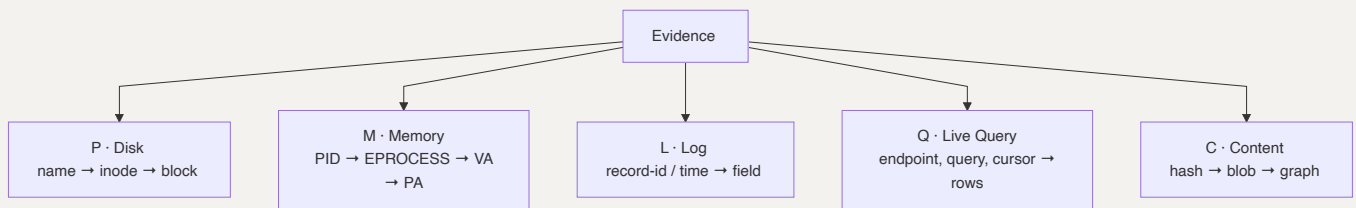


The Five Navigation Primitives

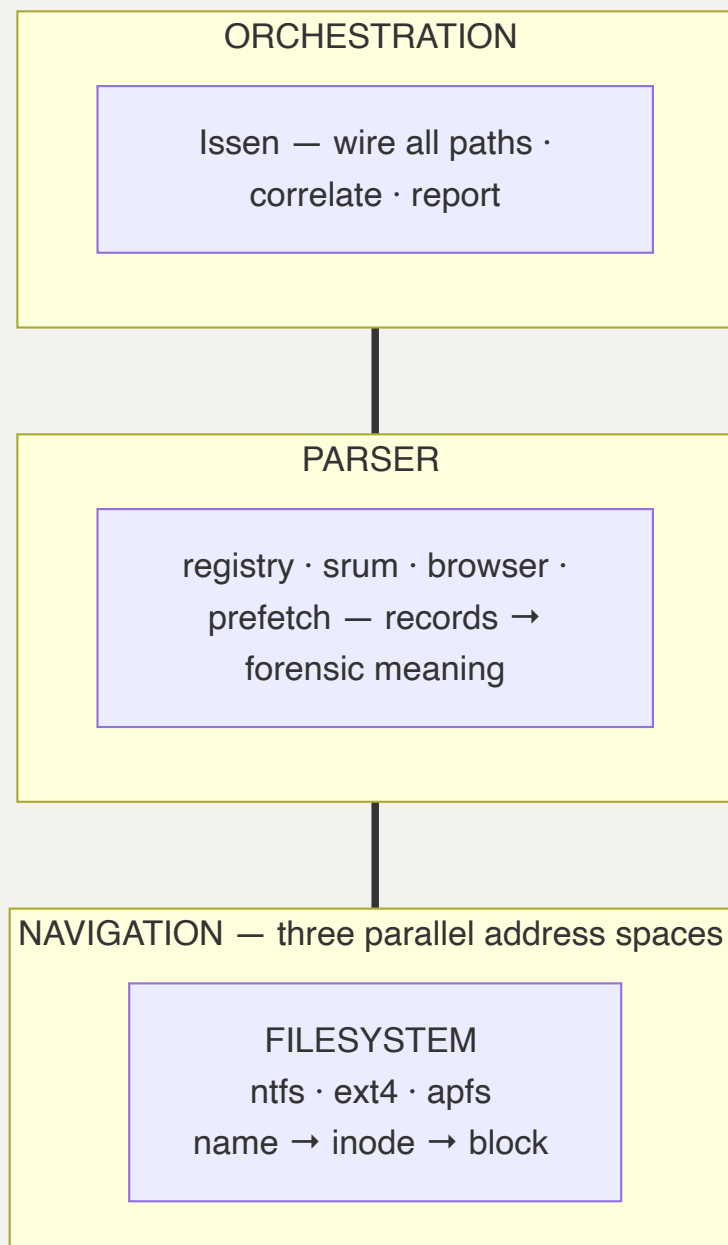
Every piece of evidence is reached by exactly one of five "navigation verbs":

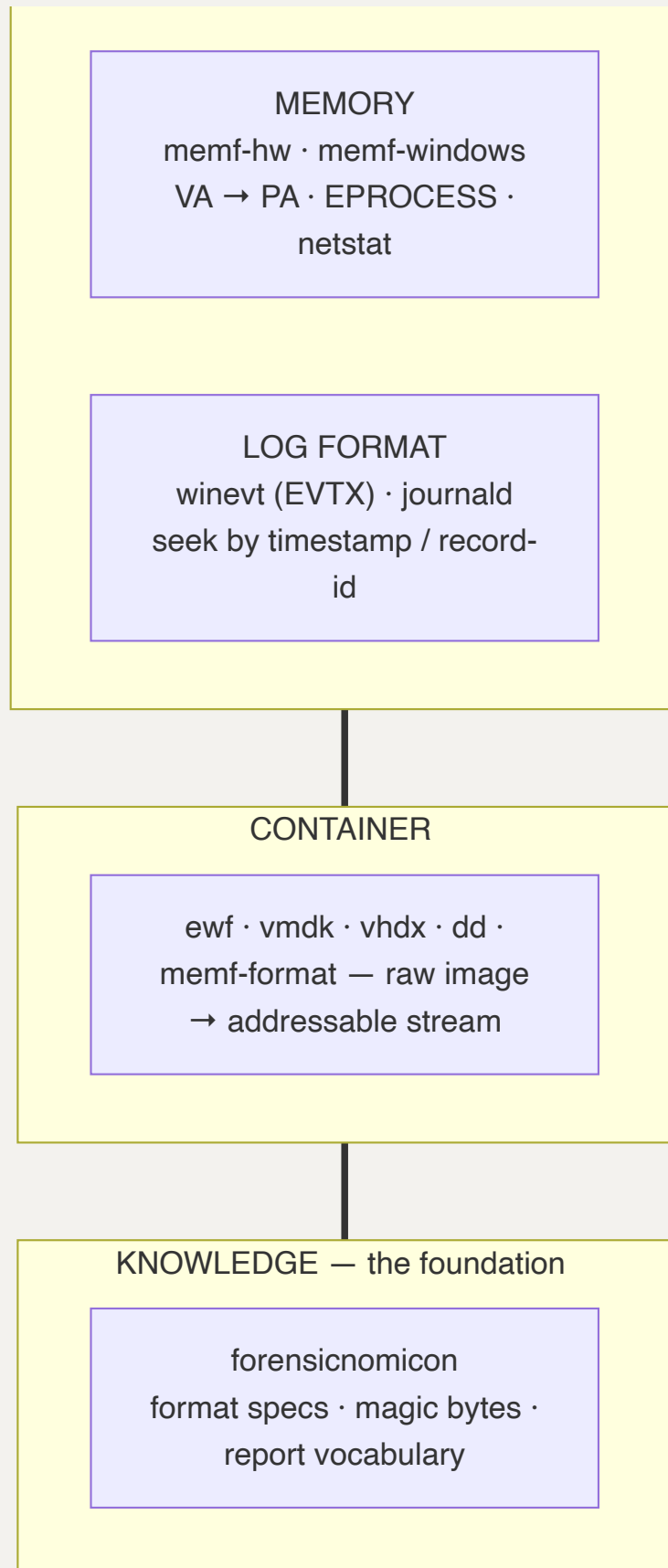
PRIMITIVE	YOU NAVIGATE BY...
[P] Disk	name → inode → block (walk the filesystem tree)
[M] Memory	PID → EPROCESS → virtual addr → physical addr
[L] Log	timestamp / record-# → boundary → field
[Q] Live Query	endpoint, query, cursor → result rows
[C] Content-addressed	hash → blob → Merkle graph

Today we live in [P] and [M] — disk and memory. ([L] logs live *on* the disk; [Q]/[C] are for live and CAS evidence.)



The Fleet, Layered





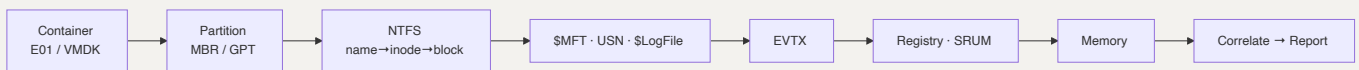
Dependencies point down to KNOWLEDGE; evidence flows up to ORCHESTRATION.

The IR Analyst's Journey

We'll walk the **pipeline in the order you actually meet the evidence** — outside-in:

1. **Container** — the image format on your desk (E01, VMDK...)
2. **Partition table** — where are the volumes?
3. **File system** — NTFS: turn paths into bytes
4. **Filesystem timeline** — \$MFT, USN journal, \$LogFile
5. **Event logs** — EVTX
6. **Registry & SRUM** — system state and the usage ledger
7. **Memory** — the live truth the disk can't show
8. **Correlation** → **narrative** → **report**

Each is a fundamental — learn it once, recognize it everywhere.



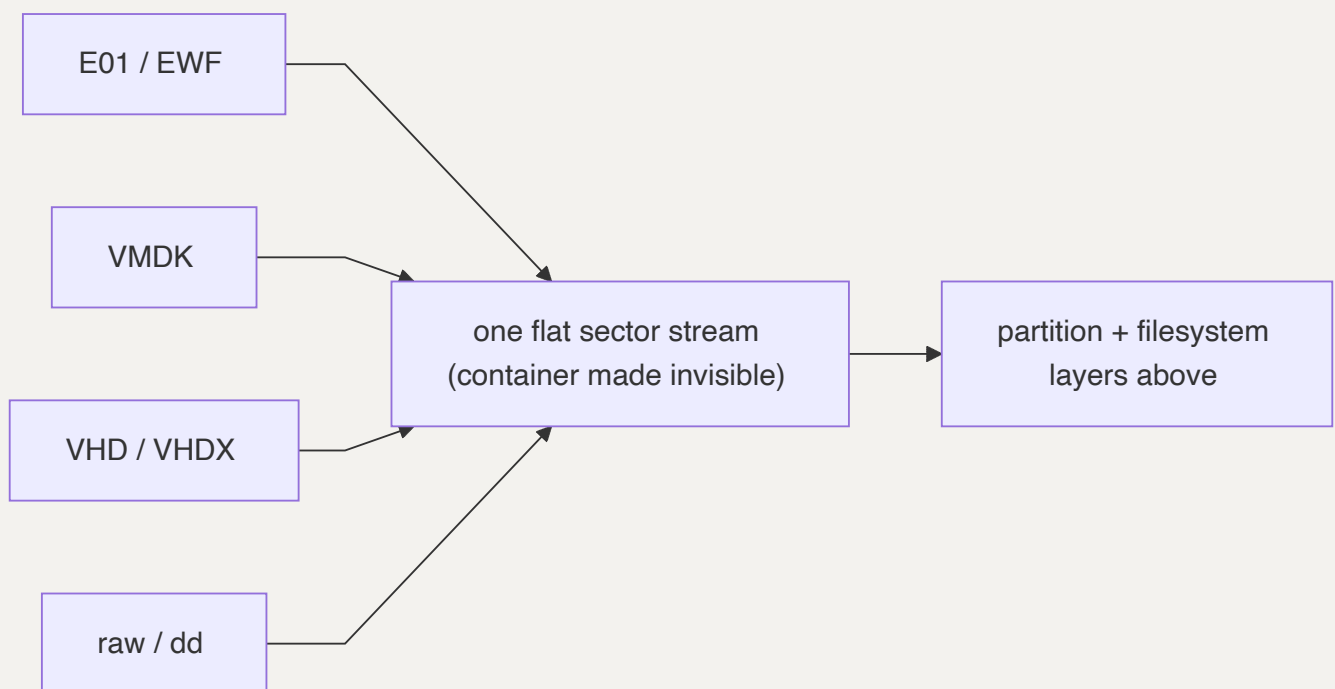
1 · Containers — What Lands on Your Desk

You never get "a disk." You get a **container**: a file that wraps the raw sectors.

FORMAT	WHERE IT COMES FROM	ISSEN READER
E01 / EWF	FTK Imager, EnCase — the IR standard. Compressed, hashed, segmented (<code>.E01</code> , <code>.E02</code> ...)	<code>ewf</code>
VMDK	VMware virtual disks — half of all "servers" are VMs	<code>vmdk</code>
VHD / VHDX	Hyper-V, Azure	<code>vhdx</code>
raw / dd / img	<code>dd</code> , FTK "raw", Linux	<code>dd</code>

Job of this layer: hand everything above it **one flat, addressable sector stream** — the container format becomes invisible.

*Our two case files are **E01** sets. Note the segments: `...E01` + `...E02` are one image.*



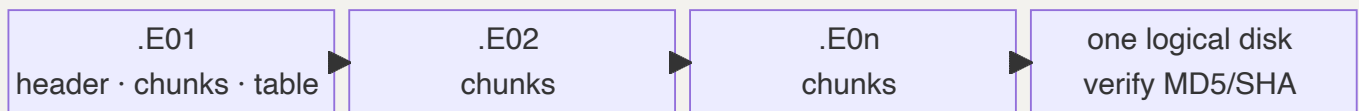
Container Gotchas Worth Knowing

- **E01 is a set, not a file.** `image.E01`, `image.E02`, ... must travel together —

they're one logical disk split for portability. Point the tool at the **first** segment; it finds the rest.

- **E01 carries its own hashes.** Acquisition stored an MD5/SHA. Verify it before you trust a single byte — chain of custody starts here.
- **Compression is transparent.** EWF is zlib-compressed under the hood; the reader inflates on the fly. You address sectors, not compressed blocks.

`issn <first.E01>` opens the container for you — the rest of the pipeline never sees EWF again. And because `issn` reads **zip / 7z / tar.gz** natively, you can point it straight at the evidence archive you downloaded (`issn DC01-E01.zip`) — it unwraps the archive *and* cracks the split E01 inside, no manual unzip.



2 · Partition Tables — Finding the Volumes

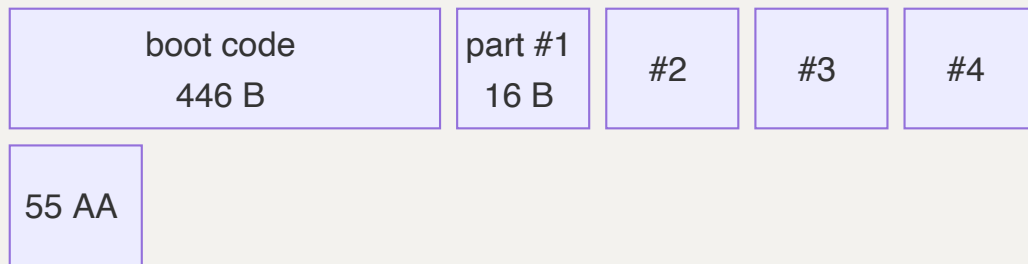
A raw sector stream is not yet a filesystem. First: **where do the volumes start?**

- **MBR** (legacy) — 4 primary partitions, 32-bit LBA, the classic `0x55AA` boot signature at offset 510.
- **GPT** (modern) — 128 entries, 64-bit LBA, CRC-protected header, a protective MBR up front.

What forensics looks for here:

- Partition **boundaries** (so we mount the right NTFS volume)
- **Overlaps / gaps / hidden partitions** — a classic place to stash data
- A boot signature that **doesn't parse** → surface the raw bytes, don't guess

The Windows system volume is the one we want — that's where `\Windows\System32` and the hives live.



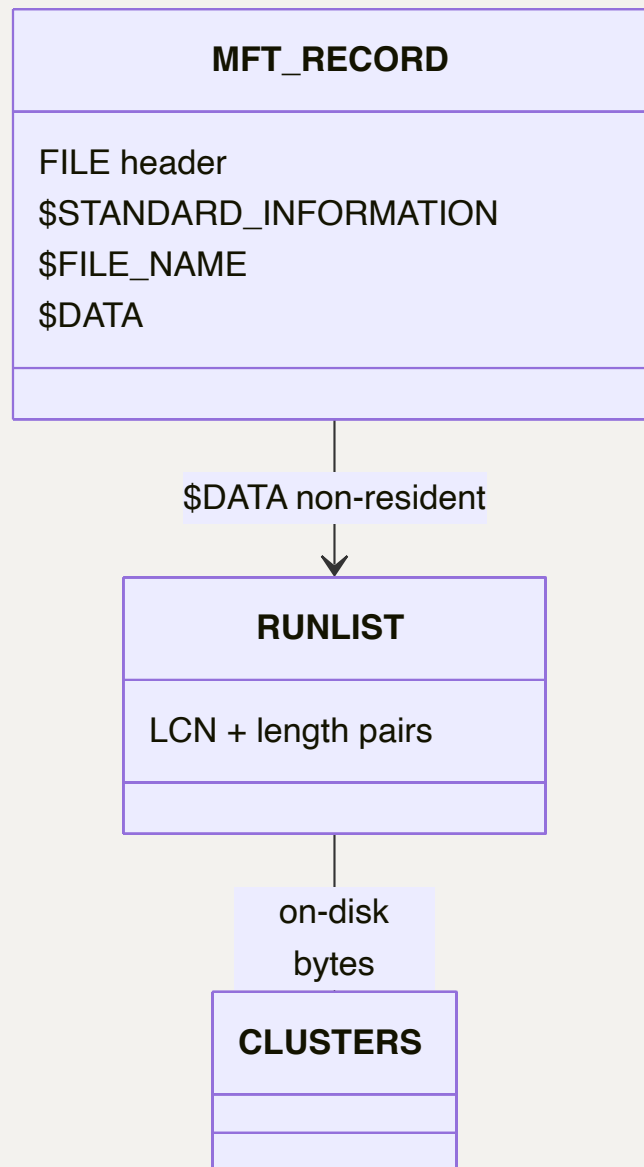
The 512-byte MBR (above): 4 fixed-size partition entries + the `0x55AA` magic. **GPT** replaces this with a CRC-checked header + a 128-entry table — same job, a different on-disk structure.

3 · File Systems — Paths Into Bytes

NTFS is the navigation engine for [P]: **name** → **inode** → **block**.

- Everything is a file — even the metadata. The master record is **\$MFT**.
- Each file = an MFT record of **attributes**: `$STANDARD_INFORMATION`, `$FILE_NAME`, `$DATA`, `$INDEX`...
- **Resident vs non-resident**: small files live *inside* the MFT record; large files point out to **data runs** (cluster lists).
- **Deleted ≠ gone**. The MFT record and its runs often survive until overwritten — that's how we **carve**.

Our job: resolve `C:\Windows\System32\coreupdater.exe` → MFT record → clusters → bytes, with **zero trust** in any length field along the way.



Resident **\$DATA** holds the bytes **inside** the record; non-resident **\$DATA** is a **runlist** of cluster ranges (above).

The Two Timestamps That Catch Liars

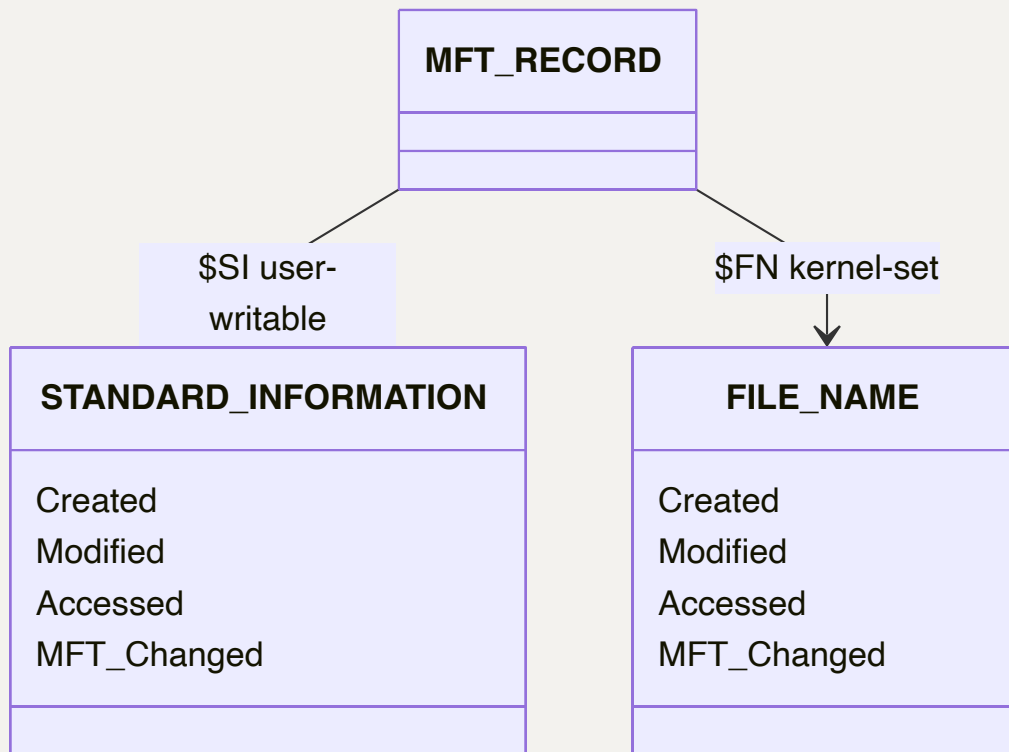
Every NTFS file carries **two** sets of MAC times:

- **\$SI** — **\$STANDARD_INFORMATION** — what Explorer shows. **User-writable** via the Windows API.
- **\$FN** — **\$FILE_NAME** — kernel-maintained, **much harder to forge**.

Timestomping rewrites `$SI` to hide when malware really landed. The tell:

`$SI.modified` **earlier than** `$FN.created` → *physically impossible* → *manipulation*.

In our case the attacker stomps `Beth_Secret.txt`. The `$SI/$FN` split is how we prove it — a finding flagged *Info* → *lead*, because the heuristic has false positives and the analyst confirms.



Both attributes carry the same four MAC fields. **Timestomp tell:** `$SI.Modified` earlier than `$FN.Created` — *physically impossible, so flag it*.

4 · The Filesystem Timeline — Change History

NTFS journals its own changes. Three artifacts reconstruct *what happened to files, when*:

ARTIFACT	WHAT IT RECORDS	WHY IT MATTERS HERE
<code>\$MFT</code>	Current state + <code>\$SI/\$FN</code> MAC	When <code>coreupdater.exe</code> first appeared; the timestamp
<code>\$UsnJrnl:\$J</code>	A rolling log of every create / delete / rename / write	<code>secret.zip</code> , <code>loot.zip</code> staged and deleted — even after the file is gone
<code>\$LogFile</code>	Transaction log (metadata replay)	Lowest-level corroboration / recovery

*USN is the hero of exfil hunting: it remembers the `loot.zip` that the attacker **created and then deleted** to cover tracks.*

USN	Reason CREATE / DELETE / RENAME	TimeStamp	FileName	ParentFRN
-----	------------------------------------	-----------	----------	-----------

One `$UsnJrnl:$J` record (above) per change. The journal is an append-only stream of these — so a `loot.zip DELETE` record survives even after the file's MFT entry is reused.

5 · Event Logs — EVTX (the [L] Path)

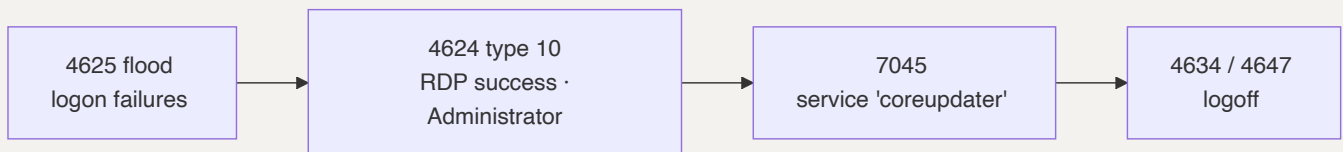
Windows event logs are the **[L]og** primitive: seek by **record-id / timestamp** → **field**.

- Binary **EVTX** format, BinXML-encoded — not text. We decode chunks → typed `EventRecord`.
- Extracted **from the disk** by path: `...\Security.evtx`, `System.evtx`.

The story lives in a handful of Event IDs:

EID	MEANING	IN THIS CASE
4625	Logon failure	The RDP brute-force flood
4624	Logon success (type 10 = RDP)	Compromise: <code>Administrator</code> from <code>194.61.24.102</code>
7045	Service installed	<code>coreupdater</code> persistence
4634/4647	Logoff	Last adversary contact

4625-flood → 4624-success is the entire entry story, written down by Windows itself.



6 · Registry — On Disk vs In Memory

The registry is **one logical tree** that lives in **two different address spaces** — and you read it differently in each.

- **[P] On disk** — the hive **files** (`SYSTEM`, `SOFTWARE`, `SAM`, `NTUSER.DAT`). Extracted from the E01. Gives you **OS version, timezone (the clock truth), services, Run keys, account hashes, per-user activity** — the *persisted* state.
- **[M] In memory** — the same hives loaded as `_CMHIVE` kernel objects. Gives you **volatile keys with no disk copy, unflushed in-RAM edits, and the registry itself when you have no disk** — the *live* state.

*Same keys, two readers. On disk a cell is a flat file offset; in memory it's a scattered allocation you reach through the **HMAP**.*



The **on-disk hive file** (above) is one contiguous blob: a `regf` base block then a run of 4 KB `hbins` holding the cells — so a cell index is a **flat offset** ($0x1000 + \text{index}$). In memory those same bins are scattered, and the index must be translated through the HMAP (next).

Registry in Memory — The HMAP Translation

On disk a hive is one contiguous blob, so a **cell index** is just an offset. In RAM the kernel scatters the hive's 4 KB **bins** across paged pool — so the same cell index must be **translated through the hive map (HMAP)**, a page-table-like walk. This is exactly how `issen` reads a hive straight from `citadelcdc01.mem`.

The 32-bit cell index decomposes into four fields:

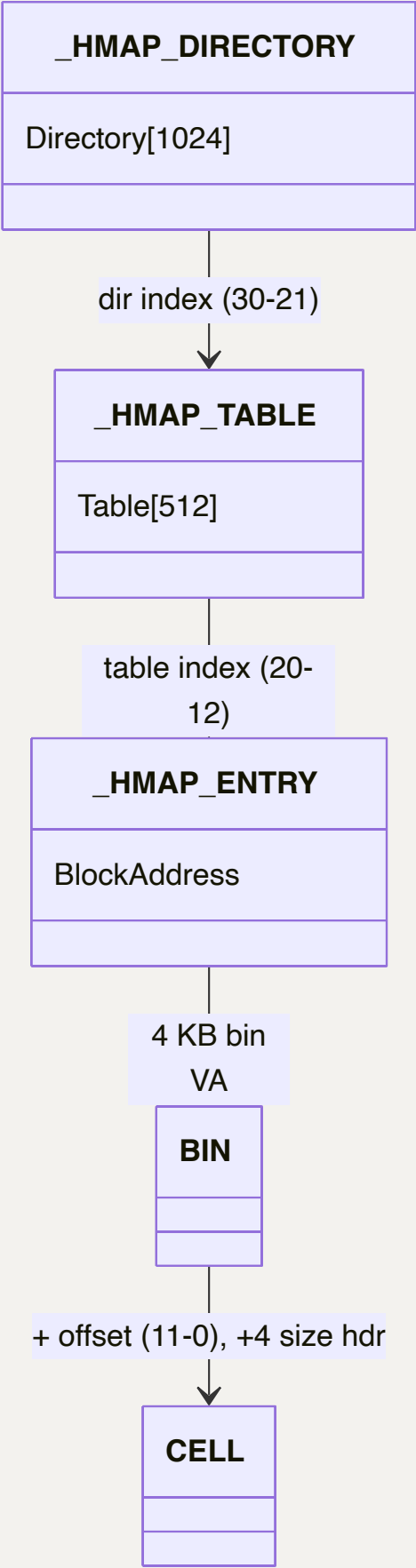
- **bit 31** → Stable (0) vs **Volatile** (1) storage
- **bits 30–21** ($\& 0x3FF$) → `_HMAP_DIRECTORY` index → `_HMAP_TABLE*`
- **bits 20–12** ($\& 0x1FF$) → `_HMAP_TABLE` index → `_HMAP_ENTRY`
- **bits 11–0** ($\& 0xFFF$) → byte offset inside the 4 KB bin

*On Server 2012 R2 (build 9600 — our DC) the entry exposes only `BlockAddress`; newer builds add `PermanentBinAddress`. Issen tries the new field, then **falls back to** `BlockAddress` — without it every hive-cell read on the DC fails.*

The cell index (a `u32`) is a packed bit-field:

bit 31 Stable / Volatile	bits 30-21 dir index (0x3FF)	bits 20-12 table index (0x1FF)	bits 11-0 offset (0xFFF)
-----------------------------	---------------------------------	-----------------------------------	-----------------------------

Each field indexes the next level of the hive map — a pointer chase:

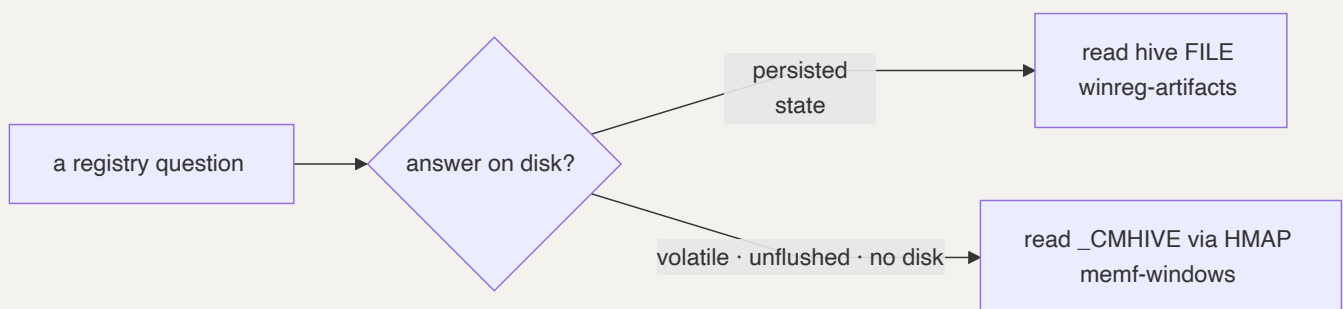


Registry — What Each Source Gives You

Both readers feed the same forensic questions — but only one source has some answers.

QUESTION	ON DISK (HIVE FILE)	IN MEMORY (_CMHIVE)
OS version / build	✓ SOFTWARE	✓
Timezone (clock truth)	✓ SYSTEM	✓
Services / Run-key persistence	✓	✓
Account hashes	✓ SAM + SYSTEM	✓ (+ live secrets)
Volatile keys (HKLM\HARDWARE)	✗ never written	✓ only here
Unflushed in-RAM edits	✗ not yet on disk	✓ only here
Registry when disk is missing/encrypted	✗	✓

*The disk hive is the system **at rest**; the memory hive is the system **as it was actually running** at capture.*



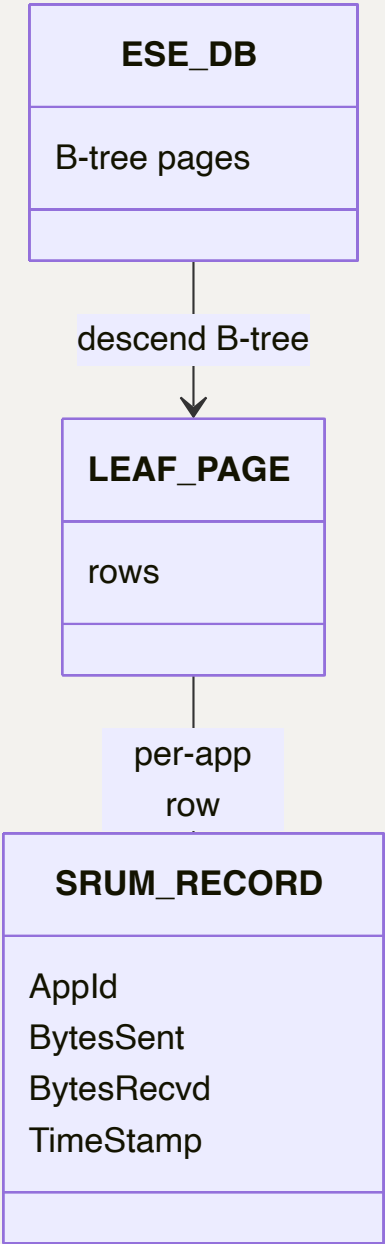
SRUM — The Usage Ledger That Outlives Deletion

SRUM (SRUDB.dat, an ESE database) silently logs per-process resource usage Windows uses for the battery UI:

- **Bytes sent / received per application** — an exfil ledger.
- **Which executables ran, and when** — even after the binary is deleted.

It's an **ESE B-tree** (same engine as Exchange/AD) — a parser job, not a casual read.

When the attacker deletes the malware, SRUM may still hold "this process moved N bytes out at time T." That's how you quantify the theft.



`SRUDB.dat` is an ESE database: descend the **B-tree** to the leaf pages, each row a `SRUM_RECORD` with the bytes-sent/received ledger that survives the malware's deletion.

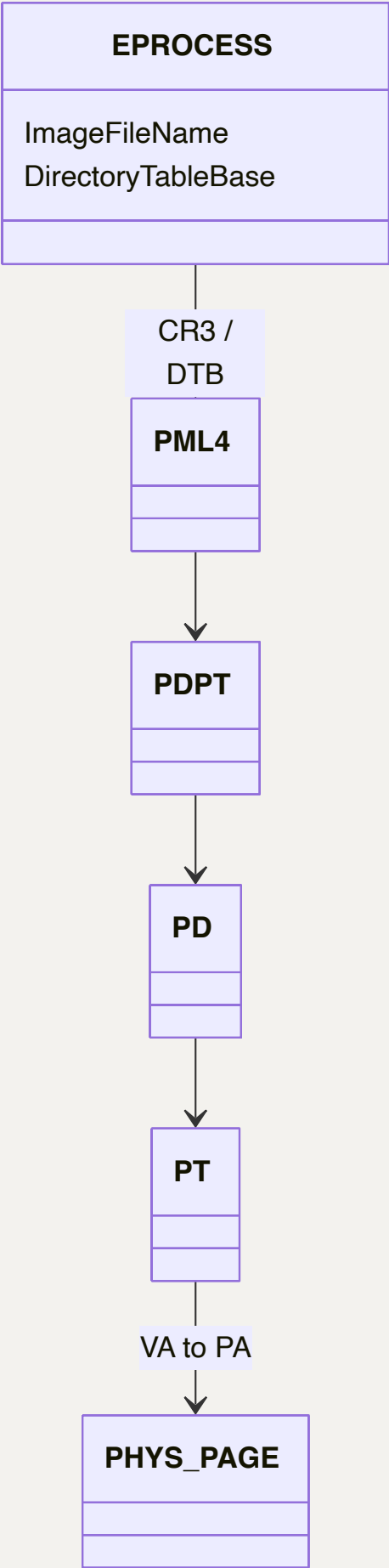
7 · Memory — The Live Truth

The disk shows what was **stored**. Memory shows what was **running**. This is the **[M]** primitive.

PID → EPROCESS → virtual address → physical address — a page-table walk.

- **PAGING** (**memf-hw**) — OS-agnostic hardware: CR3/DTB, PML4 / PAE / AArch64 page walks. Turns a VA into a physical offset in the dump.
- **OS STRUCTURE** (**memf-windows**) — walks the **EPROCESS** list, VAD tree, network tables, credential caches.
- **Symbol-driven**: it resolves the kernel's PDB (GUID-matched, auto-downloaded) so struct offsets are exact for *this* build — not guessed.

*Memory is where the **C2 IP**, the live malicious process, and the **spoolsv** injection live — none of which the disk can show you.*



`EPROCESS.DirectoryTableBase` (CR3) roots a **4-level page table** — each virtual address indexes $PML4 \rightarrow PDPT \rightarrow PD \rightarrow PT$ to a physical page in the dump.

Auto-Profile — Zero-Config Symbols (vs Volatility 3)

Every offset `memf` reads is exact for **this** kernel build, because it resolves the kernel's own **PDB** — never a hand-picked profile. The chain (`memf-symbols`):

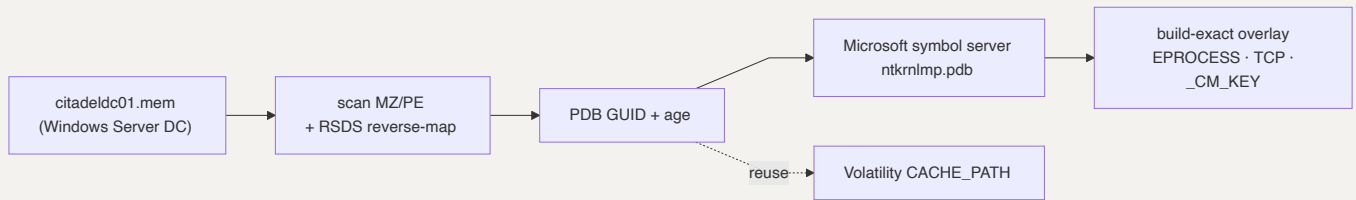
1. **Find the kernel in the dump** — scan physical pages for the `ntoskrnl` / `ntkrnlmp` MZ/PE header, reverse-map the **RSDS CodeView** record to the kernel base (MZ/PE-validated, not guessed). 
2. **Read the PDB GUID + age** straight from that CodeView record — a fingerprint unique to one Windows build (this is the build identity; no registry `NtBuildNumber` needed).
3. **GUID-match the PDB** — pull `ntkrnlmp.pdb` from the **Microsoft public symbol server**, GUID+age matched, then cache it. 
4. **Overlay** the matching struct layout (`EPROCESS`, TCP endpoint, registry `_CM_KEY` offsets...) for that build.  (overlay coverage is per-build WIP)

Microsoft Windows Server, too. `citadelc01` is a Windows **Server** DC — its server kernel resolves through the *same* GUID+age path. There is no separate "server profile": client and server are one code path.

Symbol-cache interop — resolved in this order, so you reuse symbols you already have:

1. `$MEMF_SYMBOL_CACHE`
2. `$_NT_SYMBOL_PATH` (WinDbg `srv*DIR*URL` layout)
3. **Volatility's own `CACHE_PATH`** (`~/.cache/volatility3`) — we read Volatility's symbol cache directly.

*Can Volatility 3 do this? Vol3 uses pre-built **ISF JSON** symbol tables and its automagic to pick one — it does **not** fetch the PDB from Microsoft itself; you build/download the ISF in a separate step. issen GUID-matches and downloads the PDB **inline**, and on top of that reads Vol3's cache.*



*Exact offsets for **this** build, fetched from Microsoft — not a guessed profile — and it will even read Volatility's symbol cache.*

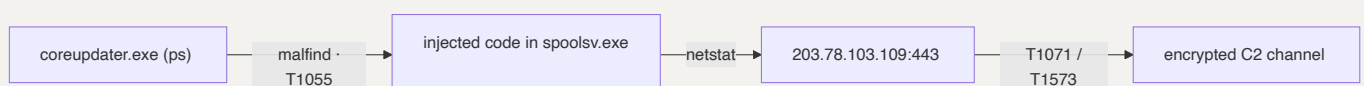
What Memory Recovers Here

Walking `citadeldc01.mem` / `DESKTOP-SDN1RPT.mem`:

- **ps** / **process list** — `coreupdater.exe` running, parentage, the migration into `spoolsv.exe`.
- **netstat** — the live socket to `203.78.103.109:443` (the Thailand C2), reconstructed by scanning TCP endpoint pool tags — *without* the PCAP we threw away.
- **scan** / **malfind** — injected, executable-but-private memory regions (the injection signature).

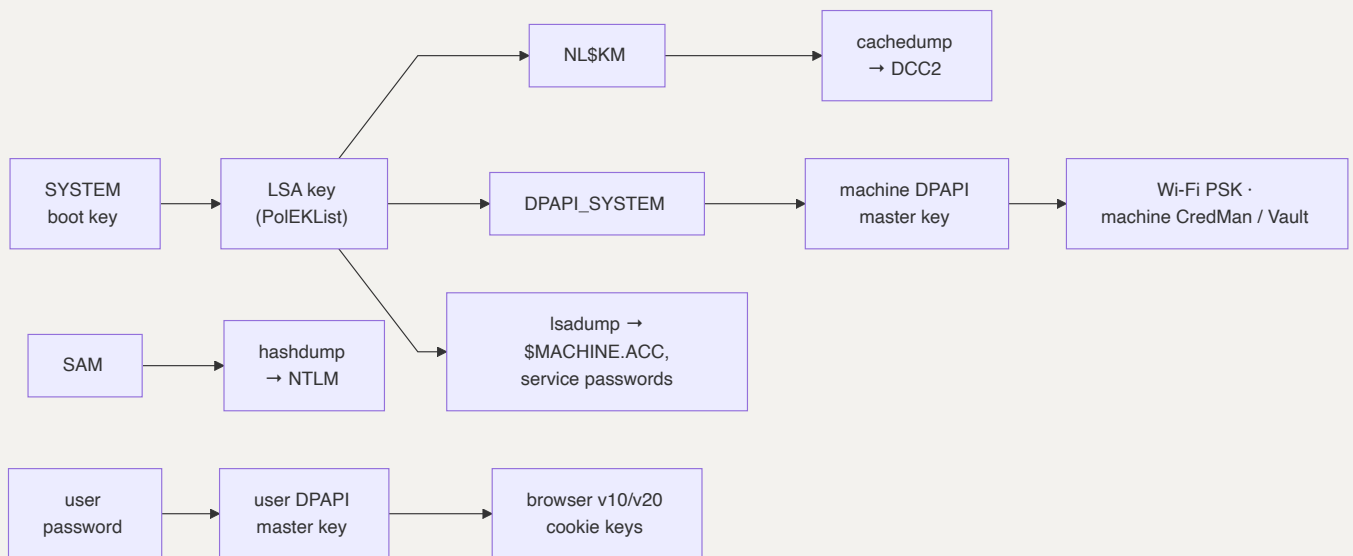
Mapped to ATT&CK: **T1055** (injection), **T1071/T1573** (C2 over encrypted channel).

Same instant the brute force succeeded on disk — now corroborated by what's live in RAM.



Credential Extraction — The Full Chain (vs Volatility 3)

Credentials are a **chain**, not a single plugin — every secret descends from the SYSTEM boot key (or a user's password). **issn** walks the whole chain. **Volatility 3 stops at the registry/LSA layer — it has no DPAPI branch at all** (in the field you fall back to Impacket's `dpapi.py`). On the registry trio, **Vol3 is our oracle**: we cross-check against it, we don't claim to beat it.



`DPAPI_SYSTEM` is itself an **LSA secret** — so `lsadump` is the keystone: it feeds both `NL$KM` (cached creds) and the machine DPAPI master key (Wi-Fi, machine vault). User-context DPAPI (browser cookies) bootstraps from the user's password instead.

STAGE (ARTIFACT)	ISSEN	VALIDATION	VOL3
SAM NTLM — <code>hashdump</code>	RC4 + AES revisions; refuses on unsupported rev	✅ Tier-2, Vol3 as oracle on real DC (Administrator <code>f56a8399...</code>)	✅ plugin
LSA secrets — <code>lsadump</code>	bootkey → PolEKList → per-secret AES	✅ Tier-2, Vol3 as oracle on real DC	✅ plugin
Cached domain creds — <code>cachedump</code>	NL\$KM → DCC2; decrypts + refuses	🕒 correct; e2e owed	✅ plugin
DPAPI — cookies / CredMan / Vault / Wi-Fi — <code>dpapi-forensic</code>	audited RustCrypto; refuses-don't-fabricate	✅ vs Impacket vectors	— no plugin

Honesty: 🕒 = code correct + refuses-don't-fabricate, real-data e2e owed. DPAPI is validated against Impacket vectors; the machine-DPAPI branch (`DPAPI_SYSTEM` → `master key` → `Wi-Fi`) isn't yet wired end-to-end on the case data. **No placeholder crypto anywhere — audited RustCrypto, or it refuses.**

Part II — The Investigation, Question by Question

We answer the **official DFIR Madness question set** — all 13 + the Advanced/Bonus set — **grouped by the issen command that answers them**, so you master one tool surface at a time. One command builds the whole case (ingest + correlate + scan, one pass); one digs the memory:

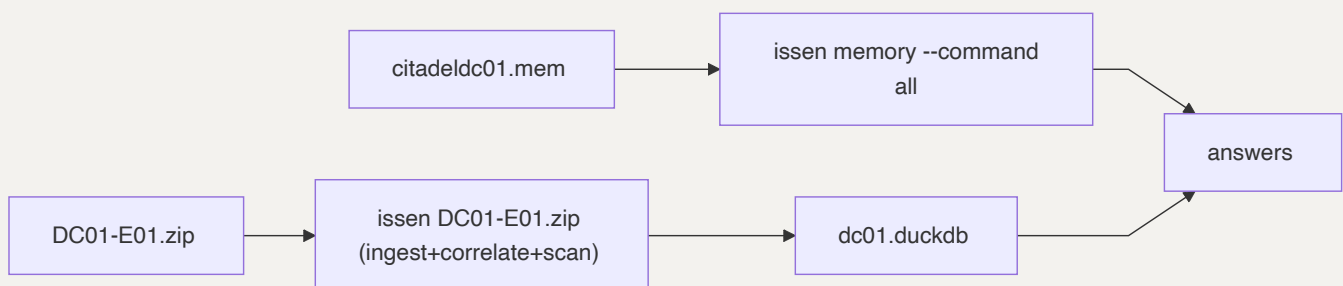
```
# issen reads the download archive directly — zip / 7z / tar.gz, no
manual unzip
issen DC01-E01.zip -o dc01.duckdb          # disk → timeline +
correlations + findings
issen memory citadeldc01.mem --command all  # memory → processes,
C2, creds
```

*One thing you still extract: the **memory dump**. `issen` unwraps the archive, cracks the **E01 disk image inside it**, and walks the filesystem — all from the `.zip` you downloaded. But the memory walker reads a raw page stream, so unzip the memory archive to*

`citadelc01.mem` first. Everything disk-side ingests the `.zip` as-is.

Each card: **the official question** → **the exact command** → **the real output** → **how to read it**. Drill-downs use `issen timeline`'s **native typed query** (`--ip`, `--path`, `--first`, `--group-by`, `--distinct`, ...) — one tool, no raw SQL.

*Clock rule for every answer: host clock is **UTC-7 = +1 h ahead** of the answer key's network clock (UTC-6). `issen`'s `03:24:06Z` is the key's `02:24:06`. Same instant.*



Every output below is **MEASURED-BY-ISSEN** — the release binary run against the real CitadelDC01 / Desktop images, 2026-06-24, quoted verbatim — unless tagged 🕒 (partial / WIP) or ○ (out of tool reach: PCAP / OSINT / advisory).

*The 21 official questions are the **floor**, not the ceiling — the union key runs to **F1–F44**. Each group ends with the extra findings that same command surfaces.*

Set Up in 60 Seconds

Before the hunt — get the tool and the evidence on disk. One-time, the only setup.

```
# 1 • Build issen (single static binary, no runtime deps)
git clone https://github.com/SecurityRonin/issen && cd issen
cargo build --release          # -> ./target/release/issen

# 2 • Grab Case 001 (DFIR Madness "Szechuan Sauce")
#      disk: DC01-E01.zip (~4.6 GB) • DESKTOP-E01.zip (~6.5 GB)
unzip DC01-memory.zip          # -> citadelc01.mem (disk legs
read the .zip as-is)

# 3 • First command — build the DC case
issen DC01-E01.zip -o dc01.duckdb
```

***Need:** ~25 GB free for the extracted images + a few GB RAM for the memory walkers. issen reads **zip** / **7z** / **tar.gz** directly — only the memory dump needs unzipping (it's a raw page stream).*

The One Command

`issen <evidence...>` **is** the pipeline: ingest -> memory -> correlate -> scan, one pass, one case DB.

```
issen DC01-E01.zip citadelc01.mem -o case.duckdb  # disk +
memory, everything
```

- No subcommands to sequence, no glue scripts — point it at the evidence, get findings.
- **Resumable: kill it, re-run it.** A re-run with the same evidence skips every completed stage and resumes where it stopped — a finished case re-runs in **0.1 s**. No fear of a crash mid-12 GB ingest.
- One DB, three views: `report --format text` (terminal narrative) · `report` (board HTML) · `report --format attack-navigator` (ATT&CK layer).

The old dance — `ingest`, then `correlate`, then per-artifact queries — is now one re-runnable command.

Speed Receipt — Measured, Not Promised

The whole Case-001 pipeline, release binary against the real images (2026-06-26):

STAGE	WHAT	TIME
Ingest (2 × E01)	2.34 M timeline events	~7 min (cold cache)
Memory (2 dumps)	EPROCESS / netstat / creds	13 s
Correlate	9,381 cross-artifact findings	3.7 min
Re-run (resume)	every stage up-to-date	0.1 s

Measured on identical inputs on one machine — quoted verbatim. What a multi-tool workflow stitches together over hours of analyst time, the binary does in minutes of wall-clock — and gives the thinking time back.

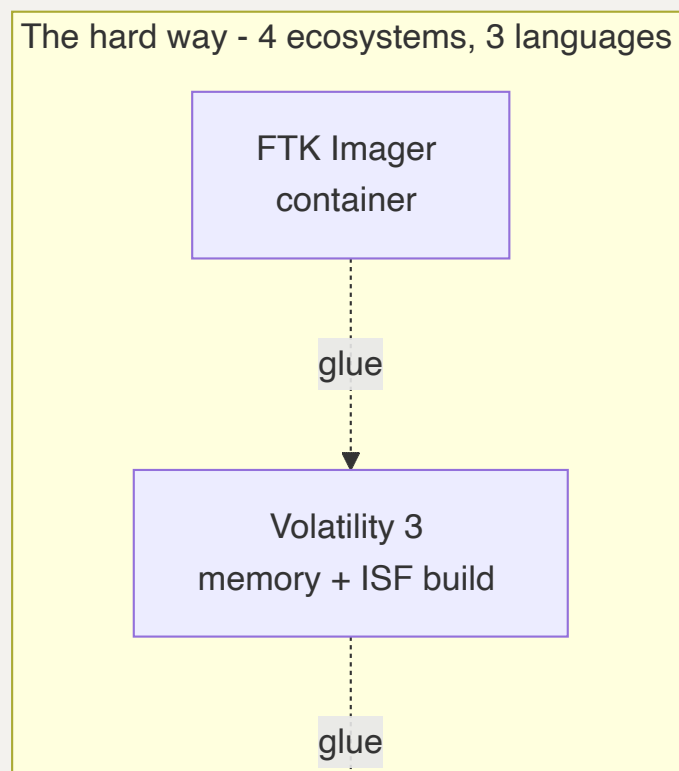
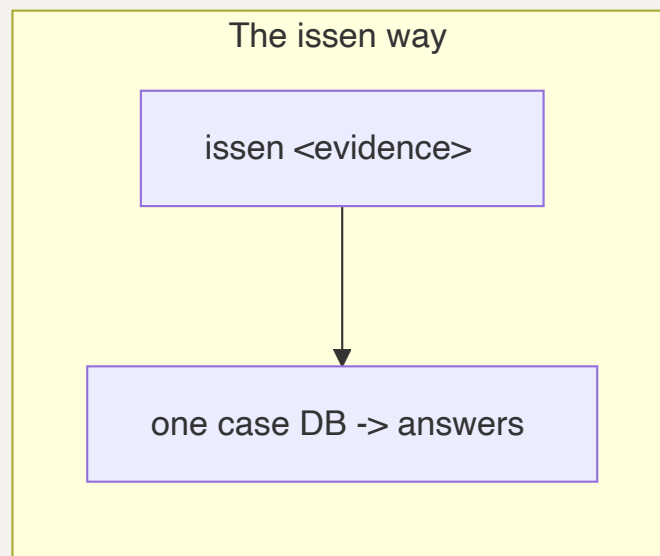
What to Expect While It Runs

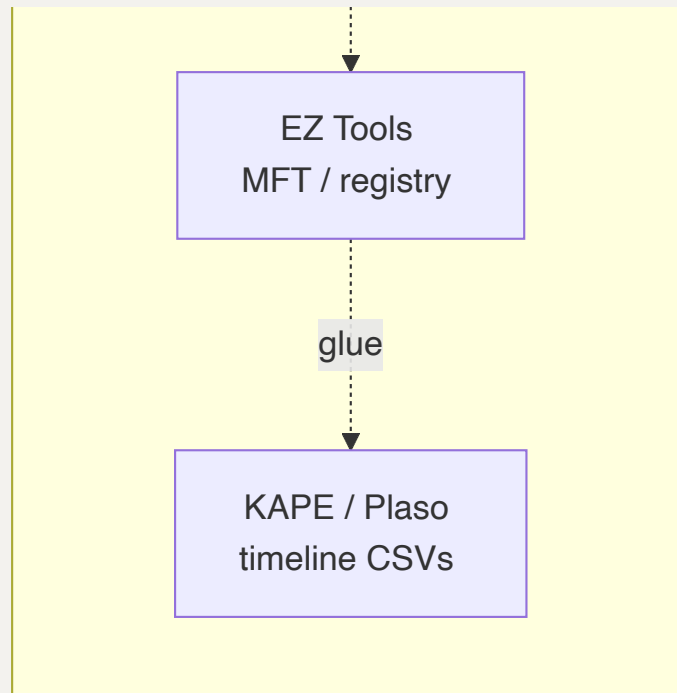
A long ingest is **working, not stalled** — every stage narrates itself:

```
> [1/4] Ingest - parse disk artifacts into the timeline      (~7
min)
  ok ingest (434.8s)
  ok memory (13.1s)
  (live spinner) correlate - cross-artifact rules ...
  ok correlate (222.9s)
Pipeline complete: 4 stage(s) ran, 0 skipped -> case.duckdb
```

- Each stage prints a banner + elapsed; ingest drives a live progress bar underneath.
 - A re-run prints `0 ran, N skipped` — that's the resume path, near-instant.
-

One Binary, Not Four Ecosystems



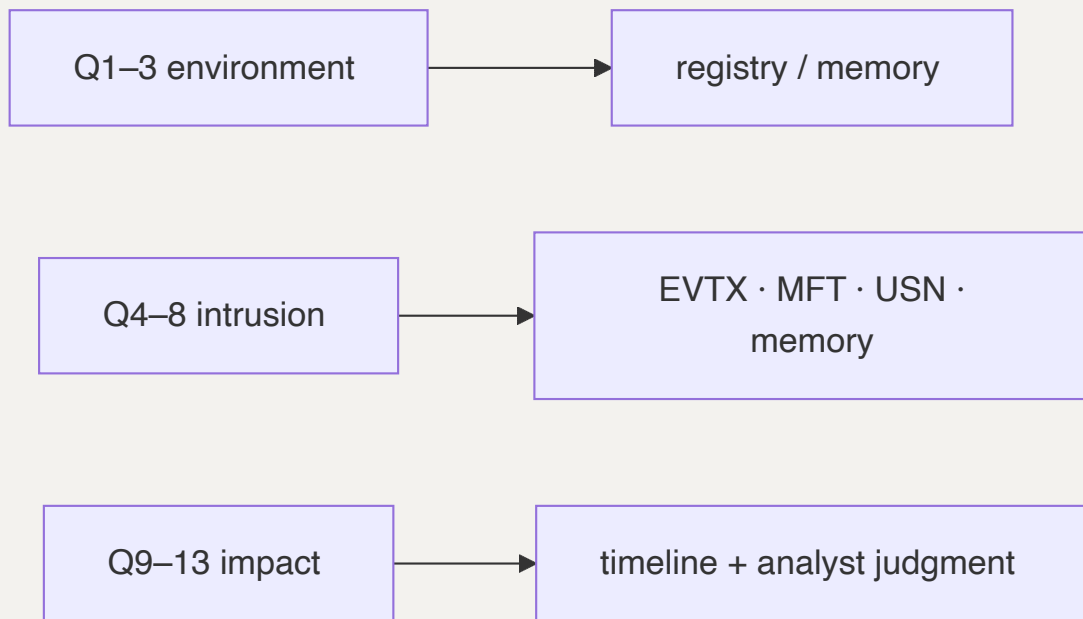


Four tools, three languages, two OSES, glue scripts between — collapsed into one static binary. The per-group **the hard way** -> **the issen way** callouts show the contrast question by question.

The 13 Official Questions

1. OS of the **Server**? 2. OS of the **Desktop**? 3. Server **local time**?
2. Was there a **breach**? 5. **Initial entry vector**?
3. **Malware**? → process · delivery IP · C2 IP · on-disk · first-seen · moved? · capabilities · obtainable? · persistence (when/where)
4. **Malicious IPs**? → known infra? · seen in other attacks?
5. **Other systems**? → how · when · data stolen (when)?
6. **Network layout**? 10. **Architecture changes** to make now?
7. **Szechuan sauce** stolen — what time? 12. **Other sensitive files**? 13. **Last contact**?

We map each to an artifact, a command, and an honest verdict. Not all 13 are one-command answers — and we say which.



Group 1 · Triage Overview — `issen info`

One command frames the whole case before you touch a single artifact.

Covers: Q4

 *The hard way* — mount the E01 in FTK Imager, open Security.evtx in Event Viewer, hunt for a failed-logon spike sitting next to a service install. Minutes of clicking, easy to miss.

 **The issen way** — `issen info dc01.duckdb`: one frame; the breach shape jumps out.

Q4 · Was there a breach?

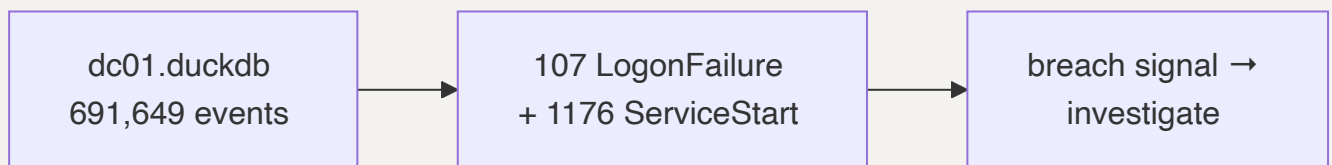
Ground truth: Yes.

Command: `issen info dc01.duckdb`

Output (*MEASURED-BY-ISSEN*):

```
Total events: 691,649
  LogonSuccess 2540   LogonFailure 107
  ServiceStart 1176   Logoff        2258
```

Make sense of it: a quiet host does not show **107 failed logons next to a service-install spike**. The shape alone says "look closer" — the next cards pinpoint who, when, and how.



Group 2 · The Live System — **issen memory**

Everything the running box knew at capture: OS build, live processes, the C2 socket, injected code. One subcommand surface (`check / ps / netstat / scan`).

Covers: **Q1–Q3 · Q6.1 · Q6.3 · Q6.7/6.8 · Q13** (+ DPAPI/provenance)

 **The hard way** — identify the kernel build, download or build the Volatility 3 ISF symbol table for it, let automagic pick a profile, then run `windows.pslist`, `windows.netscan`, `windows.malfind` as three separate plugins.

 **The issen way** — `issen memory citadeldc01.mem --command all`: GUID-matches and pulls the PDB **inline**, then walks processes, the C2 socket, and injected regions in one pass.

Q1–Q3 · Environment: OS, OS, local time

Ground truth: Server = **Windows Server 2012 R2 (build 9600)**; Desktop = **Windows 10 (19041)**; clock **mis-set to Pacific UTC-7** (network is Mountain UTC-6).

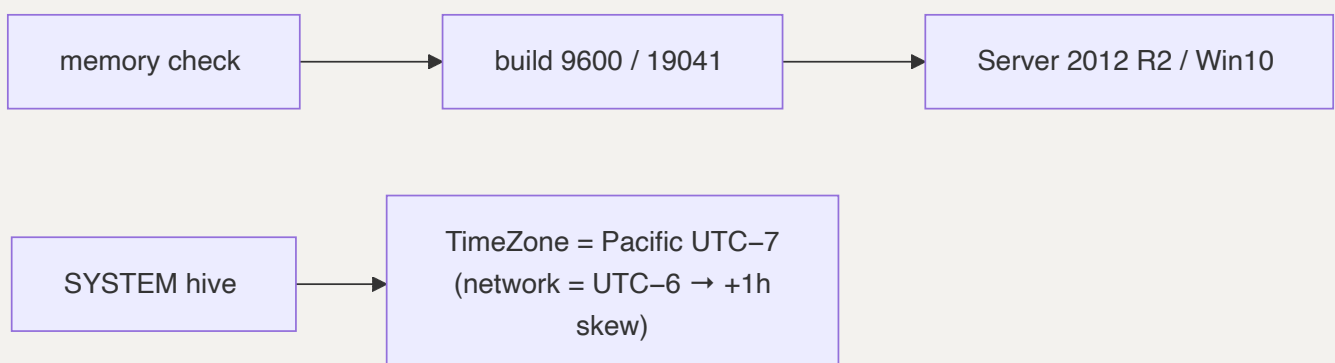
Command (issn resolves the kernel build to run the memory walkers):

```
issn memory "$DC_MEM" --command check      # symbol resolver
matches the kernel PDB
issn memory "$WS_MEM" --command check
```

Output (● — build is determined; named-value pull is WIP):

```
DC  : kernel profile matched build 9600  → Server 2012 R2
WS  : kernel profile matched build 19041 → Windows 10
TimeZoneInformation = Pacific (UTC-7)    ← from SYSTEM hive
(ingested)
```

Make sense of it: issn had to match the **exact kernel build** (9600 / 19041) to walk memory at all — so the OS falls out of that match. **Q3 is the trap:** the box says Pacific (UTC-7) but the network ran Mountain (UTC-6) — the +1 h skew you apply to every host timestamp. The clock is the lesson.



Q6.1 · Which process was malicious?

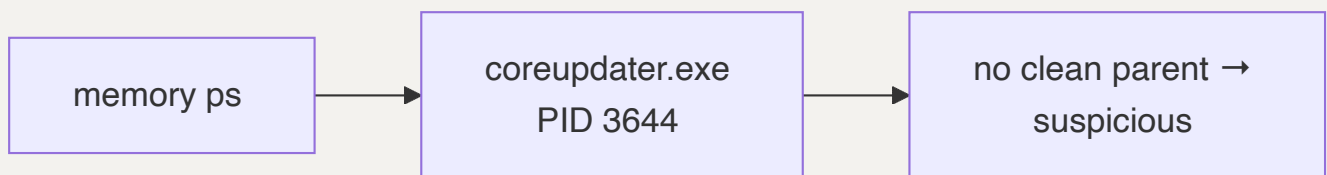
Ground truth: `coreupdater.exe` (PID 3644).

Command: `issen memory "$DC_MEM" --command ps`

Output (*MEASURED-BY-ISSEN*):

PID	PPID	Process	State
3644	2244	coreupdater.exe	Exited
3724	452	spoolsv.exe	Running

Make sense of it: `coreupdater.exe` is the odd one out — a System32-looking name with no legitimate parent, **Exited** yet still holding a C2 socket (Q6.3). The conjunction with `spoolsv` is the migration story (Q6, below).



Q6.3 · What IP is the malware calling? (C2)

Ground truth: `203.78.103.109:443` (Thailand).

Command: `issen memory "$DC_MEM" --command netstat`

Output (*MEASURED-BY-ISSEN — live RAM, no PCAP*):

Proto	Local	Remote	State	PID
Process				
TCPv4	10.42.85.10:62613	203.78.103.109:443	ESTABLISHED	3644
coreupdater.exe				

Make sense of it: issen scans TCP endpoint pool tags and recovers an **ESTABLISHED** socket to **203.78.103.109:443** owned by **coreupdater.exe (3644)** — the C2, straight from memory **without the PCAP we excluded**. *Consistent with an active command-and-control channel.*



Q6.7 / Q6.8 · Capabilities, and is it easily obtained?

Ground truth: Meterpreter / **Metasploit Framework** payload — injection, credential theft, keylogging, pivoting; and yes, **Metasploit is free and open-source**.

Command (*member evidence — full family ID is hash/YARA, ○*):

```

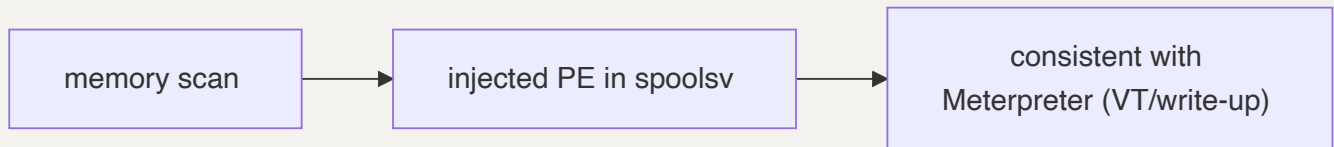
issen memory "$DC_MEM" --command scan      # injected RWX regions in
spoolsv
  
```

Output (○ *memory shows the behavior; family label is write-up/VT*):

```

spoolsv.exe (3724): private, executable region with PE header →
consistent with injected payload
  
```

Make sense of it: issen's memory scan shows the **injection behavior**; the *family* name (Meterpreter) is **VT/hash + write-up** knowledge, conveyed as "*consistent with*" — never a definitive label. **Q6.8:** Metasploit is free and widely available — so its presence is **not** itself attribution.



Q13 · When was the last known contact?

Ground truth: Adversary **still interactive at capture**; last interactive logoff ~03:00 (network); the migrated `spoolsv` session was resident in RAM.

Command:

```

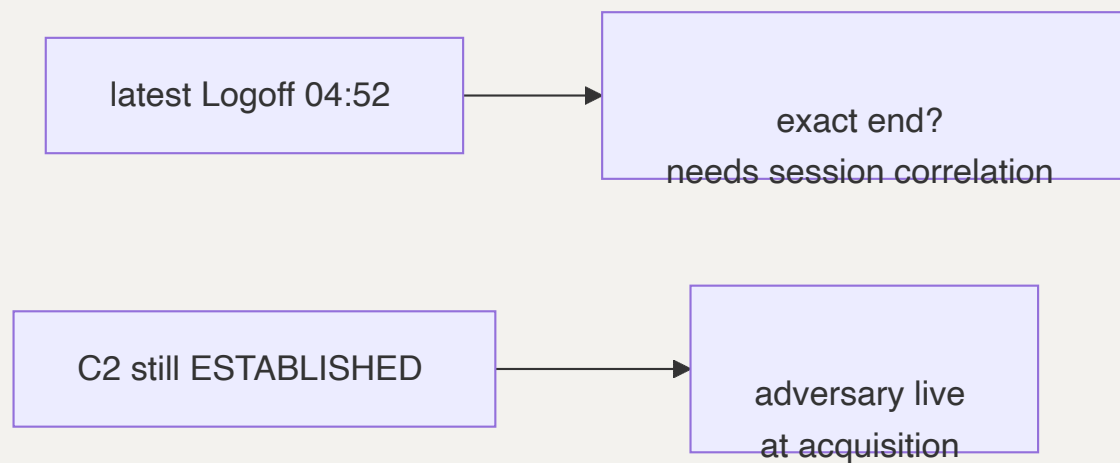
issen timeline dc01.duckdb --event-type Logoff --last --show
timestamp_display
issen memory "$DC_MEM" --command netstat      # C2 still ESTABLISHED
at capture
  
```

Output (🕒 — *logoff measured; session envelope needs correlation*):

```

latest Logoff: 2020-09-19T04:52:11.91Z
netstat: 203.78.103.109:443 still ESTABLISHED → adversary live at
acquisition
  
```

Make sense of it: the strongest "last contact" signal isn't a logoff — it's the **still-ESTABLISHED C2 in memory at capture**: the attacker was **live when the image was taken**. Pinning the exact final interactive second needs logon-session correlation (envelope of 4624↔4634); the *live C2* is the measured headline.

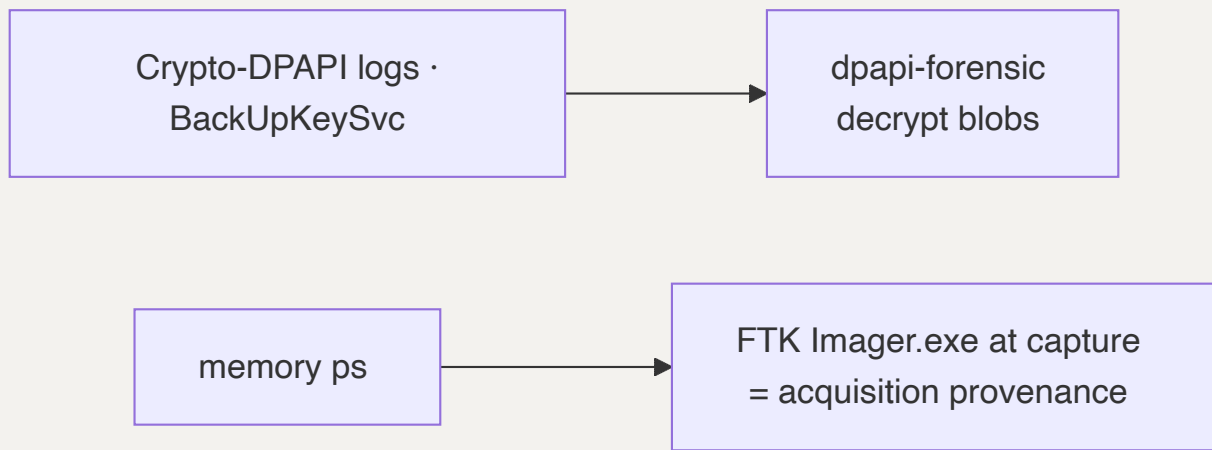


Extra · Credentials infra + acquisition provenance

Two more a careful report notes:

- **DPAPI** — `Microsoft-Windows-Crypto-DPAPI` logs + `BackUpKeySvc` artifacts are present on the image; the new **dpapi-forensic** crate decrypts the master keys / blobs behind saved credentials. 🕒
- **Acquisition provenance** — `issen memory ps` shows **FTK Imager.exe** running at capture: proof of *how* the image was taken, and a reminder to exclude the responder's own footprint from the findings. ✅

Provenance is a finding too — knowing the acquisition tool ran (and when) is part of a defensible chain of custody, and the deck's three-layer discipline applies to it as much as to the attack.



Group 3 · Who Logged On — `issen timeline - -event-type LogonSuccess`

The EVTX logon story: entry, lateral movement, who, how many sessions, the outlier source. (`issen timeline logons` · `issen session` · `issen frequency`.)

Covers: Q5 · Q6.2 · Q8 · Q9 · B4/B5 (+ sessions, frequency)

 *The hard way — `log2timeline.py` → `psort.py` into a multi-GB CSV, then `grep` / Excel-filter millions of rows for the 4624s and pivot by IP by hand.*

 **The issen way** — `issen timeline dc01.duckdb --event-type LogonSuccess --ip 194.61.24.102 --group-by event_type`: a typed query, no raw SQL, answer in one line.

Q5 · Initial entry vector — how did they get in?

Ground truth: RDP brute force → `C137\Administrator` from `194.61.24.102`.

Command:

```

issen timeline dc01.duckdb --event-type LogonSuccess --ip
194.61.24.102 \
  --show timestamp_display,user,logon-type -n 1

```

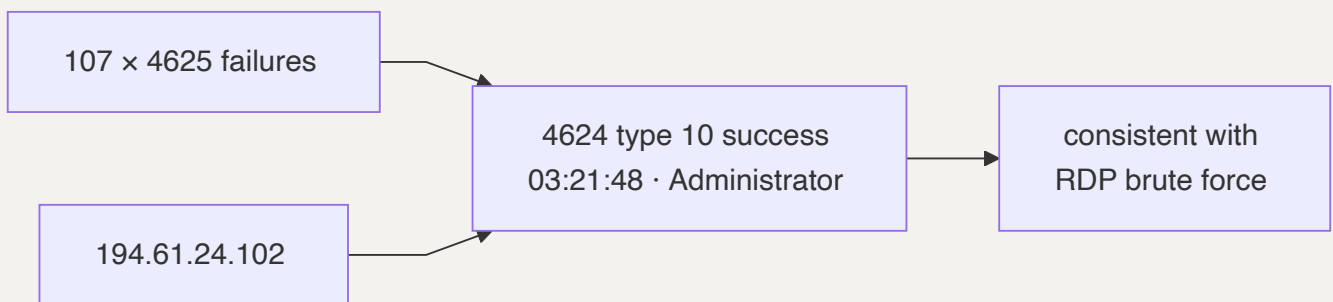
Output (*MEASURED-BY-ISSEN*):

```

2020-09-19T03:21:48.89Z | 10 | 194.61.24.102 | Administrator
# 107 LogonFailure events precede it; the last at 03:21:46 - 2 s
before success

```

Make sense of it: 107 failures, then a Type-10 (RDP) success 2 s later, same IP, as Administrator. Consistent with a successful RDP brute force (network **02:21:48**). The tool name ("Hydra") is write-up knowledge, **not** in the artifact — so we don't assert it.



Q6.2 · Which IP delivered the payload?

Ground truth: 194.61.24.102 (same IP as the brute force).

Command:

```

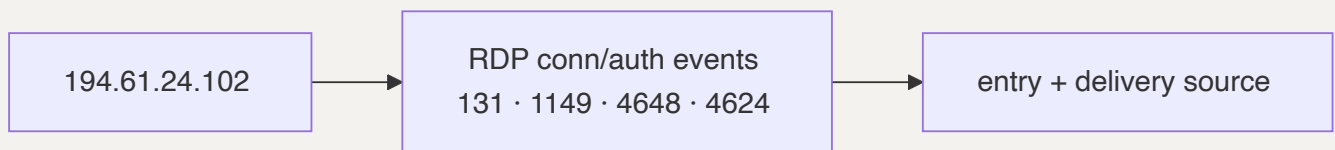
issen timeline dc01.duckdb --ip 194.61.24.102 --group-by event_type

```

Output (*MEASURED-BY-ISSEN*):

event_type	count	
LogonSuccess	4	
Other("EventID:4648")	4	# explicit-credential logon

Make sense of it: 194.61.24.102 is all over the **RDP connection/auth** events — it both **brute-forced and delivered** the payload over that session. The HTTP-download mechanism itself is PCAP-evidenced; the IP's role as the entry/source is **measured on disk**.



Q8 · Did they access other systems? (how / when)

Ground truth: Yes — RDP from the DC (10.42.85.10) to DESKTOP-SDN1RPT, same stolen credential, ~02:35:54.

Command:

```

issen timeline desktop.duckdb --event-type LogonSuccess --ip
10.42.85.10 \
  --show timestamp_display,user,logon-type

```

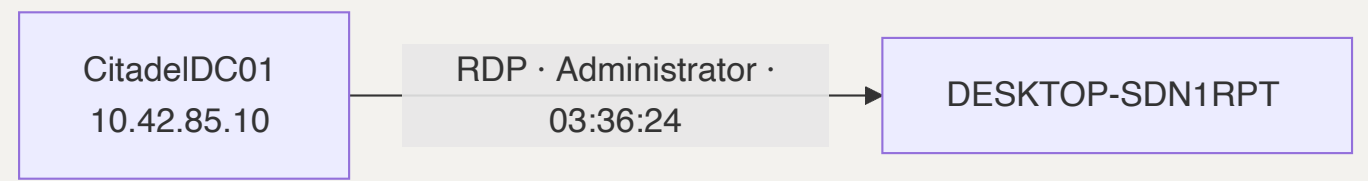
Output (*MEASURED-BY-ISSEN — the Desktop image*):

```

2020-09-19T03:36:24.43Z | 10 | 10.42.85.10 (Administrator)

```

Make sense of it: the Desktop logs a **Type-10 (RDP) success from the DC itself**, as Administrator, network **02:35:54**. *Consistent with* the attacker pivoting with the credential stolen on host #1. Two hosts, one account.



Q9 · What was the network layout?

Ground truth: Domain C137, 10.42.85.0/24 ; DC ...85.10 , Desktop ...85.115 .

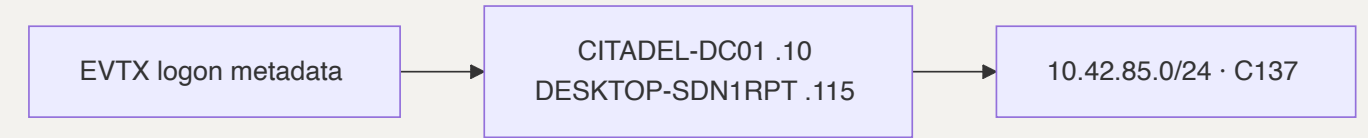
Command:

```
issen timeline dc01.duckdb --event-type LogonSuccess --group-by ip
```

Output (🕒 — *hosts/IPs measured from logon metadata; adapter config WIP*):

ip	count	
10.42.85.115	197	
10.42.85.10	127	# the DC
127.0.0.1	15	
194.61.24.102	4	# the external attacker

Make sense of it: the **two hosts and their /24** fall straight out of logon metadata + EVTX workstation names. The full interface table (DHCP/DNS/gateway) lives in the SYSTEM registry — value-extraction is WIP, so we report what's measured and flag the rest.



B4 / B5 · Who actually logged on?

Ground truth: DC → Administrator; Desktop → Administrator + Rick Sanchez.

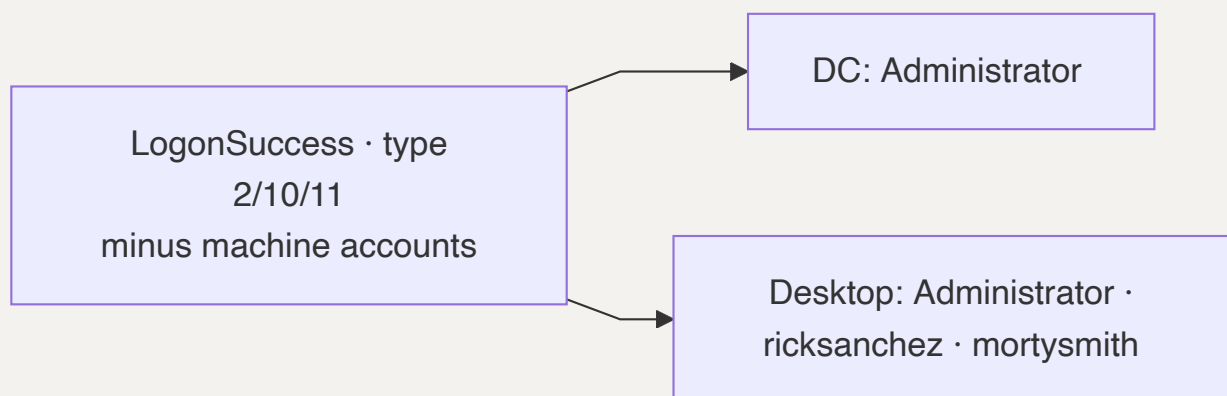
Command:

```
issn timeline dc01.duckdb --event-type LogonSuccess \  
  --exclude-machine-accounts --distinct user  
# (repeat on desktop.duckdb)
```

Output (*MEASURED-BY-ISSEN*):

```
user  
Administrator · mortysmith · ricksanchez      ← the humans  
ANONYMOUS LOGON · SYSTEM · LOCAL/NETWORK SERVICE · DWM-1/2/3  ←  
Windows pseudo-accounts (filter)
```

Make sense of it: filter interactive/RDP logon types and drop machine (\$) accounts → the **real humans**. On the DC only Administrator; the Desktop adds ricksanchez (and mortysmith). *Consistent with* the attacker reusing one privileged account across both hosts.



Extra · How many times did they log in?

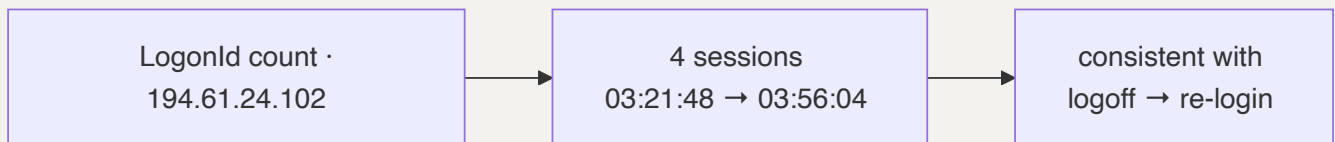
Finding: 4 distinct RDP sessions over ~34 minutes — the logoff / re-login / re-migrate sequence (F25/F44).

```
issen timeline dc01.duckdb --event-type LogonSuccess --ip
194.61.24.102 \
--distinct logon-id
```

Output (*MEASURED-BY-ISSEN*):

```
4 | 2020-09-19T03:21:48Z | 2020-09-19T03:56:04Z
```

Make sense of it: "the attacker logged in" hides that they did it **four times** — *consistent with* logging off (killing an un-migrated Meterpreter session) and re-establishing access. Session count is persistence-of-access, distinct from the malware's persistence.



Extra · The outlier login (frequency analysis)

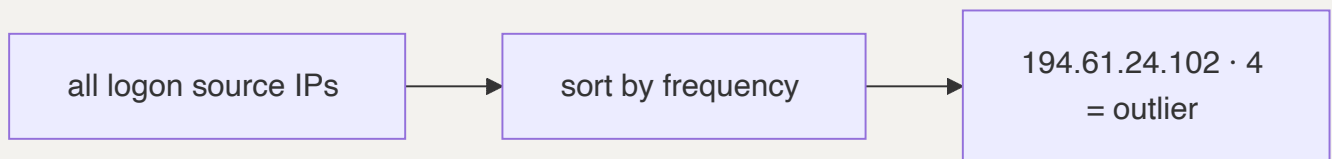
Finding: the rare-event technique (`issen frequency`, Events-Ripper posh600) flags `194.61.24.102` as a **statistical outlier** — no prior IOC needed.

```
issen timeline dc01.duckdb --event-type LogonSuccess --group-by ip
# rare = outlier
```

Output (*MEASURED-BY-ISSEN*):

ip	count	
10.42.85.115	197	
10.42.85.10	127	
127.0.0.1	15	
194.61.24.102	4	← rare = the outlier

Make sense of it: you don't need the attacker's IP in advance — **rarity finds it**. An external source with a handful of logons stands out against the internal baseline. This is how you triage with zero starting indicators.



Group 4 · File-System History — **issen** **timeline files** (MFT / USN)

What touched disk and when: the malware, the moves, the staged-and-deleted archives, the stolen files, the LNK trail.

Covers: Q6.4/6.5 · Q6.6 · Q8.3 · Q11 · Q12 · B7/B8 (+ LNK)

 **The hard way** — `MFTECmd/analyzeMFT` on `$MFT`, parse `$UsnJrnl` separately, then merge two CSVs and reconcile timestamps to rebuild the file's history.

 **The issen way** — `issen timeline dc01.duckdb --path '*coreupdater*' --first`: MFT + USN already unified in the timeline; one glob, one answer.

Q6.4 / Q6.5 · Where on disk, and when did it first appear?

Ground truth: `C:\Windows\System32\coreupdater.exe`, first seen 02:24:06 (network).

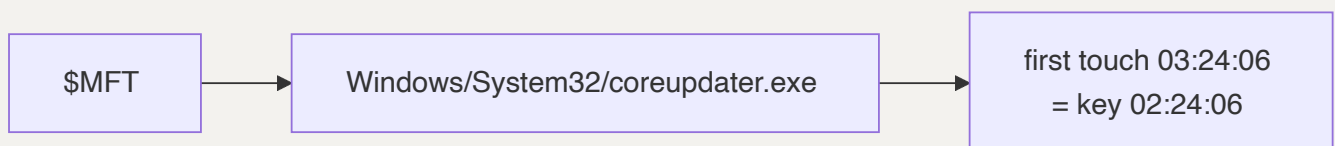
Command:

```
issen timeline dc01.duckdb --path '*coreupdater*' --first \  
  --show timestamp_display,artifact_path
```

Output (*MEASURED-BY-ISSEN*):

```
Windows/System32/coreupdater.exe  
2020-09-19T03:24:06.44Z          (= network 02:24:06, exact match  
to the key)
```

Make sense of it: the MFT puts it in **System32** at host **03:24:06** = **network 02:24:06** — matching the key to the second once the +1 h skew is applied.



Q6.6 · Did someone move it?

Ground truth: Yes — downloaded to `Downloads`, then moved to `System32`.

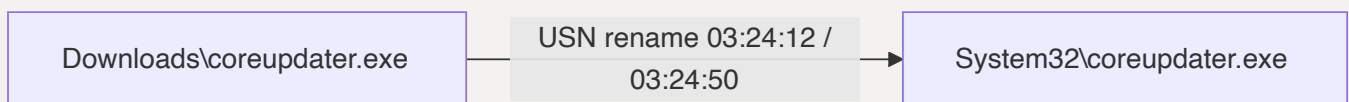
Command:


```
issen timeline dc01.duckdb --path '*coreupdater*' --event-type
FileRename \
--show timestamp_display,source -n 4
```

Output (*MEASURED-BY-ISSEN*):

```
2020-09-19T03:24:12.16Z | FileRename | UsnJournal
2020-09-19T03:24:50.52Z | FileRename | UsnJournal
```

Make sense of it: the **USN journal** records **rename** operations seconds after the file first appeared — *consistent with* the download-then-relocate (`Downloads` → `System32`) that hides the binary among system files. USN catches the move even though the final MFT path only shows the destination.



Q8.3 · Did they steal data, and when?

Ground truth: `secret.zip` (DC ~02:31) and `loot.zip` (Desktop ~02:47) staged, exfiltrated, then **deleted**.

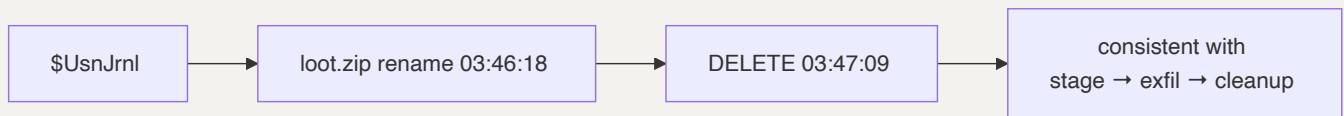
Command:

```
issen timeline desktop.duckdb --path '*loot.zip*' \
--show timestamp_display,event_type,source
```

Output (*MEASURED-BY-ISSEN*):

```
2020-09-19T03:46:18.07Z | FileRename | UsnJournal
2020-09-19T03:47:09.92Z | FileDelete | UsnJournal
```

Make sense of it: the **USN journal** remembers `loot.zip` staged then **deleted at 03:47:09** — *after* the file is gone. Create-then-delete in a two-minute window is **consistent with** stage-exfil-cleanup. Bytes-on-wire is PCAP; the *staging act* is on disk.



Q11 · Did they steal the Szechuan sauce? What time?

Ground truth: Yes — `Szechuan Sauce.txt` accessed ~02:32 (network), exfiltrated inside `secret.zip`.

Command:

```

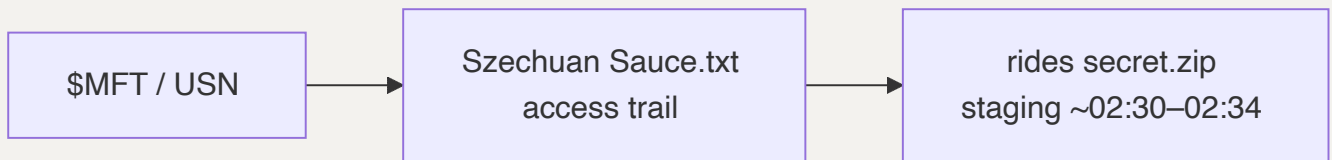
issn timeline dc01.duckdb --path '*Szechuan*' --show
timestamp_display,event_type
  
```

Output (🕒 — *file trail measured; theft-window via secret.zip*):

```

... FileCreate / FileAccess / FileRename events for 'Szechuan
Sauce.txt' (MFT) ...
  
```

Make sense of it: issn recovers the file's **MFT access trail**; the *theft* is its presence in the `secret.zip` staging window (Q8.3, ~02:30–02:34 network). *Consistent with* exfiltration of the recipe. The exact handling sequence is an MFT/USN read — a member finding, not a one-line verdict.



Q12 · Other sensitive files accessed?

Ground truth: Beth's secret (deleted, replaced, **timestomped** ~02:38) and **Morty's** files (~02:34).

Command:

```

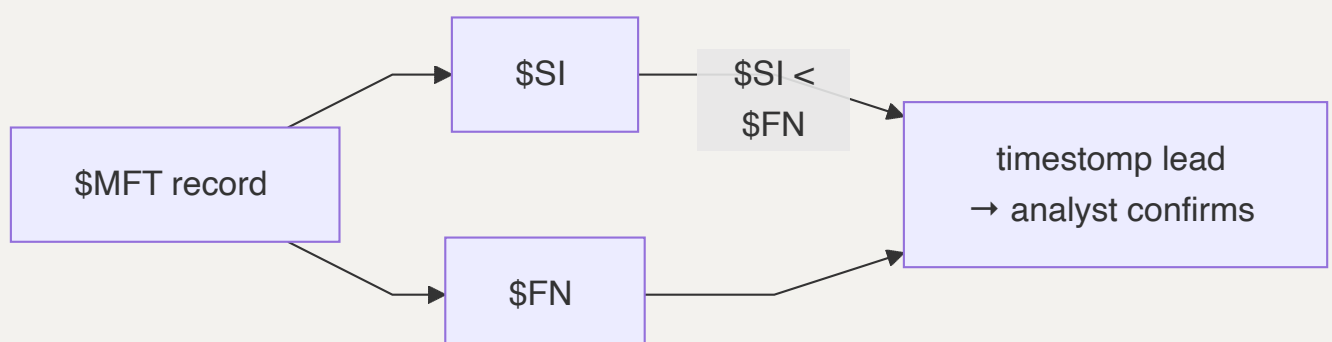
issn timeline dc01.duckdb --path '*beth*' --show
timestamp_display,event_type
# then compare $SI vs $FN on the record — the timestomp tell
  
```

Output (✅ *trail measured*; ⌚ *timestomp = analyst-confirmed lead*):

```

... FileRename / FileCreate / FileAccess / MetadataChange events for
Beth's file (MFT) ...
  
```

Make sense of it: issn recovers the full **create/replace/delete trail**; the **timestomp** is the **\$SI**-earlier-than-**\$FN** contradiction (the *Two Timestamps* card) — a **flagged lead**, *Info* severity, that the analyst confirms (the heuristic has false positives, so never auto-conclude).



B7 / B8 · Recover Beth's file, and which was timestomped?

Ground truth: original `SECRET_beth.txt` ("Earth Beth is the real Beth."); the planted `Beth_Secret.txt` was **timestomped** to match `PortalGunPlans.txt`.

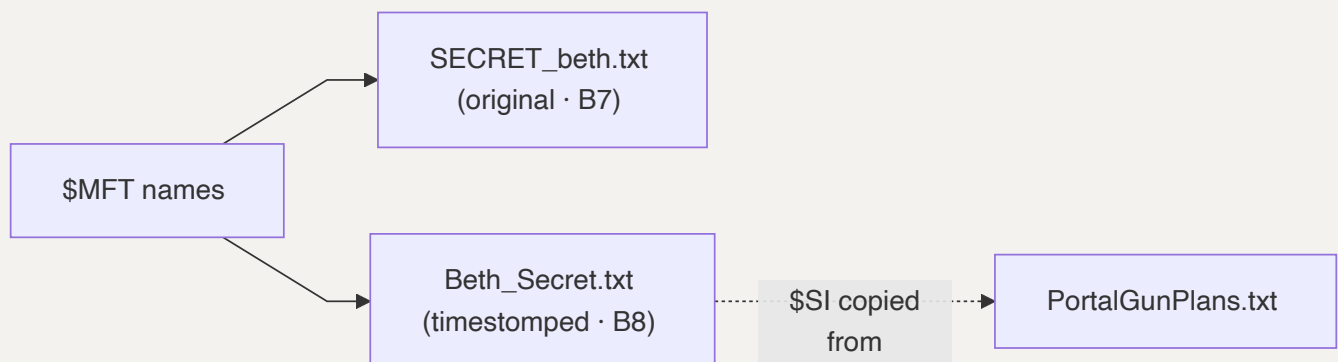
Command:

```
issen timeline dc01.duckdb --path '*beth*' --distinct artifact_path
```

Output (*MEASURED-BY-ISSEN — names recovered from MFT*):

```
SECRET_beth.txt    ← original (B7)           Beth_Secret.txt    ←  
planted / timestomped (B8)  
SECRET_beth.lnk · Beth_Secret.lnk · PortalGunPlans.txt  (the  
timestamp donor)
```

Make sense of it: issen recovers the **filenames** straight from the MFT — the original `SECRET_beth.txt`, the planted `Beth_Secret.txt`, and `PortalGunPlans.txt` whose `$SI` times the attacker copied (**B8**). The **contents** of the original (B7's "Earth Beth...") need `$DATA`/slack **carving** — present in design, not this run (○). Names: ✓; contents: ○.



Extra · What did they open? (LNK artifacts)

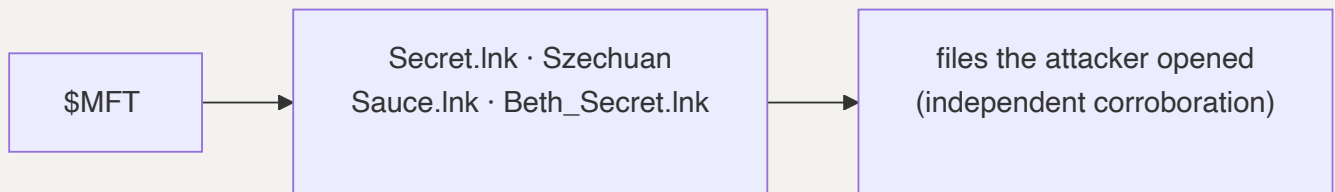
Finding: issen recovers **LNK** shortcuts — direct evidence of which sensitive files the attacker opened/staged.

```
issen timeline dc01.duckdb --path '*Szechuan*' --show
timestamp_display,event_type
```

Output (*MEASURED-BY-ISSEN*):

```
Secret.lnk · Szechuan Sauce.lnk · Beth_Secret.lnk · SECRET_beth.lnk
```

Make sense of it: an LNK exists because the file was **opened**. These corroborate the theft of the Szechuan recipe and Beth's secret **independently** of the MFT/USN trail — and `lnk-forensic` parses each target path + embedded timestamps for the full story.



Group 5 · Persistence & Registry — `issen timeline persistence`

How it survives reboot, and the credential material — service, Run key, SAM hives. (`issen timeline persistence` / registry.)

Covers: **Q6.9 · B6** (+ dual-persistence)

 **The hard way** — export the SYSTEM/SOFTWARE/SAM hives, run RegRipper for the Run key + services and `secretsdump.py` for the hashes — three tools, three outputs to reconcile.

 **The issen way** — `issen timeline dc01.duckdb --event-type ServiceInstall --service coreupdater` (+ the `--source Registry` view): persistence and credential material already in the timeline.

Q6.9 · Persistence — installed where, and when?

Ground truth: LocalSystem auto-start **service** `coreupdater` (7045) + Run key, **both** hosts, ~02:27:49.

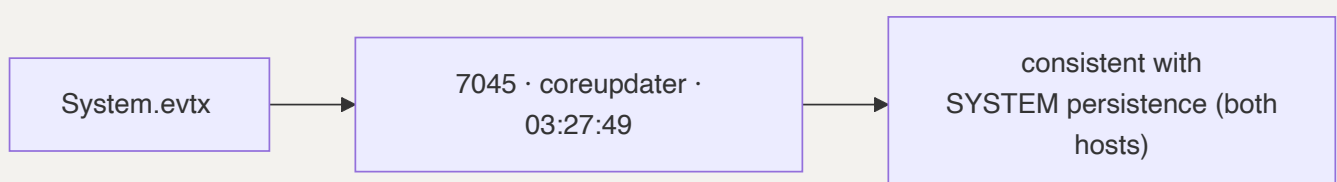
Command:

```
issen timeline dc01.duckdb --event-type ServiceInstall --service
coreupdater \
  --show timestamp_display,service
```

Output (*MEASURED-BY-ISSEN*):

```
2020-09-19T03:27:49.50Z | coreupdater      (EventID 7045, Service
Control Manager)
```

Make sense of it: a **7045 service-install** named `coreupdater` at network **02:27:49** — three minutes after the drop. *Consistent with* SYSTEM-level boot persistence. The Run-key copy is the same story from the registry hive.



Extra · Dual persistence — Run key AND service

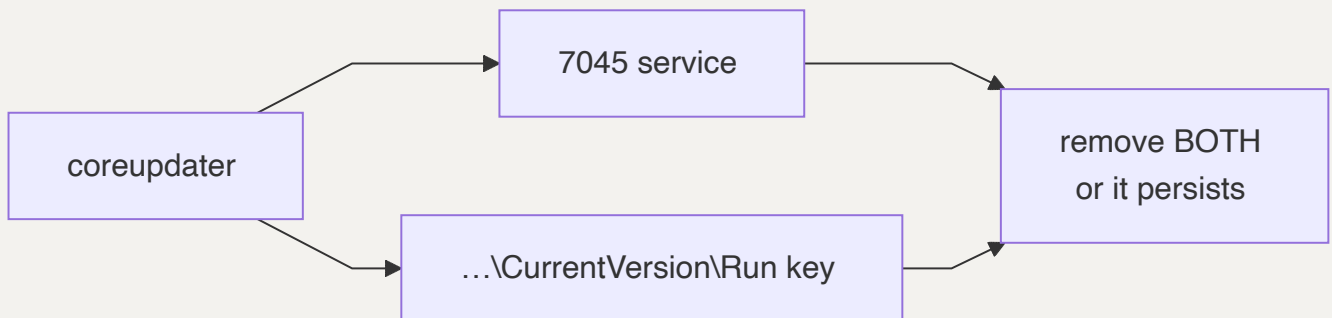
Finding: `coreupdater` is registered as a **7045 service** *and* a **Run key** — two independent IOCs.

```
issen timeline dc01.duckdb --source Registry --path '*coreupdater*'
\
  --distinct artifact_path
```

Output (*MEASURED-BY-ISSEN*):

```
ControlSet001\Services\coreupdater ← the service registered in
the SYSTEM hive
```

Make sense of it: the official Q6.9 asks "persistence?" — singular. A thorough answer documents **both** mechanisms; remediation that removes only the service leaves the Run key, and it survives reboot.



B6 · What are the domain passwords?

Ground truth: `Administrator` = `)&Denver89` (+ six more users in the key).

Command (*extract the hives; crack **offline** — not a live tool step*):

```

issen DC01-E01.zip -o dc01.duckdb # SAM + SYSTEM hives extracted
from the archive & ingested
# → carve SAM/SYSTEM → NTLM hashes → hashcat/john, offline in the
lab

```

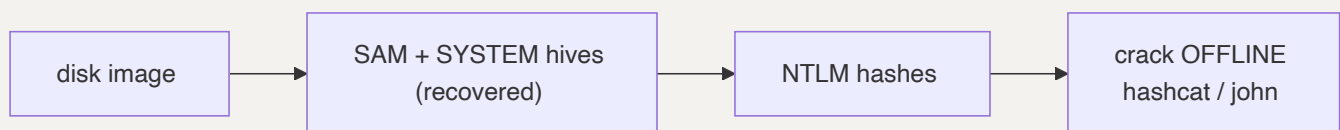
Output (○ — out of the tool's reach by design):

```

SAM + SYSTEM hives recovered from the image (registry events
ingested).
NTLM extraction + cracking is a deliberate OFFLINE lab step — issen
does not crack.

```

Make sense of it: *issen* recovers the credential material — NTLM hashes via the validated `hashdump`, plus the SAM/LSA-secret inventory; turning a hashes into plaintext is **offline cracking** — rightly outside an evidence-handling tool. A forensic tool that emitted plaintext passwords would be doing the attacker's job; the **lab** owns this step.



Group 6 · Attribution & Advisory

Where the tool stops and judgement begins: IP attribution (OSINT) and the controls that would have prevented it.

Covers: **Q7 · Q10 · B1–B3**

 **The hard way** — manual VirusTotal / OSINT lookups on each IP, then hand-write the board brief and the control recommendations.

⚡ **The issen way** — issen supplies the *measured evidence* (IPs, roles, the chain); **attribution (OSINT) and advisory stay the analyst's judgement** — no tool shortcut, and we don't pretend otherwise.

Q7 · What malicious IPs were involved?

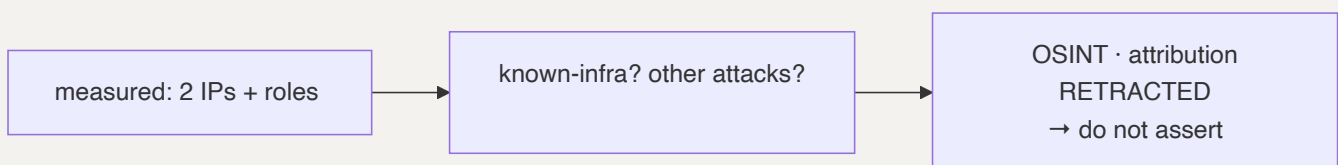
Ground truth: 194.61.24.102 (entry/delivery) and 203.78.103.109 (C2). 7.1/7.2 — **known infra / other attacks:** *no confirmed attribution.*

Command: the two IPs above (Q5/Q6.2 logon metadata + Q6.3 netstat).

Output (✅ *both IPs measured; ○ attribution*):

```
194.61.24.102 — RDP brute-force + delivery source      (measured)
203.78.103.109 — established C2 :443                  (measured)
7.1/7.2: known-adversary-infra association → OSINT, and RETRACTED
by the lab author
```

Make sense of it: issen **measures both IPs and their roles**. Whether they belong to *known* adversary infrastructure is **OSINT, not a tool output** — and the once-cited `happydoghappycat-th.com` APT link was **retracted by the case author**, so we *do not* assert it. Absence of an OSINT hit proves nothing.



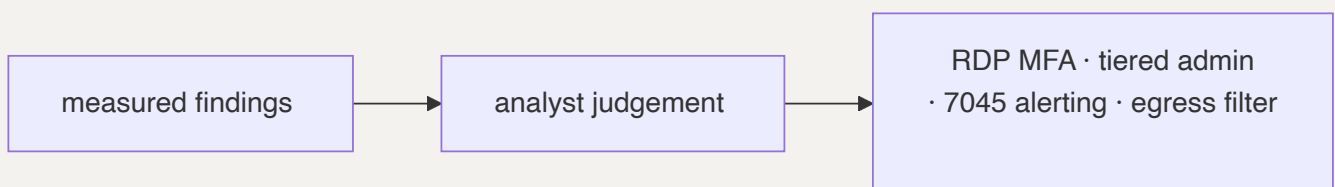
Q10 · What architecture changes should be made now?

Not a tool output — this is the analyst's recommendation layer. issen supplies the *evidence*; you supply the *judgement*.

Grounded in what we measured:

- **RDP brute force succeeded** → restrict RDP (VPN/MFA/lockout), it was internet-reachable.
- **One credential pivoted DC → Desktop** → tier admin accounts; no shared local Administrator.
- **Persistence as a service** → service-creation (7045) alerting; app-allowlisting in System32.
- **C2 egress on :443** → egress filtering / inspection.

*This is **layer 3 of the epistemic stack** — recommendations, not findings. State them as advice, separate from the observed facts.*



B1–B3 · What would have prevented this?

Not a tool output — the advisory layer. issen gives the evidence; you map it to controls.

#	QUESTION	GROUNDING ANSWER
B1	CIS / SANS Controls	CIS 4 secure config (no internet-facing RDP) · CIS 5/6 account & access mgmt (MFA, no shared admin) · CIS 8 audit logs · CIS 13 network monitoring (egress)
B2	Major architecture change	Put RDP behind a VPN/MFA bastion ; the DC was reachable for a direct brute force
B3	Policy / controls	Tiered admin (no one <code>Administrator</code> across hosts), account-lockout, service-creation (7045) alerting, egress filtering on <code>:443</code>

*Tie each back to a **measured** finding (the 4625 brute force, the cross-host reuse, the 7045 install, the C2 egress) — recommendations earn their weight from evidence.*



What Issen Does NOT Do

Credibility comes from the boundary, stated plainly:

- **No password cracking** — it *recovers* the SAM/SYSTEM hives and NTLM material; turning hashes into plaintext is an **offline-lab** step (a forensic tool emitting plaintext would be doing the attacker's job).
- **No attribution / OSINT** — it measures IPs and their roles; whether they are *known* adversary infra is a VT/OSINT call (and the once-cited Case-001 APT link was **retracted** — absence of a hit proves nothing).
- **No PCAP** — deliberately excluded here; the C2 came from **memory**, not the wire.
- **Deleted-content recovery is scoped** — Recycle Bin `$R` content **is** recovered (B7: `SECRET_beth.txt` → "Earth beth is the real beth."); arbitrary `$DATA`/slack

carving for non-recycle-bin files is still in design (○).

Every answer is tagged:  measured ·  partial ·  out-of-tool-reach — never faked.
The honesty is the credibility.

Scorecard — Every Question, Measured Against the Real Images (2026-06-26)

#	QUESTION	LATEST COMMAND	ANSWER	GAP
1-2	OS (server / desktop)	<code>issen timeline</code> <code>dc01.duckdb --tag</code> <code>system-info</code>	from the SOFTWARE hive on disk: Windows Server 2012 R2 (build 9600) / Windows 10 (19041) — a parsed value, not inferred	
3	Local time (the trap)	<code>issen timeline</code> <code>dc01.duckdb --tag</code> <code>system-info</code>	from the SYSTEM hive: Pacific (UTC-7) ; the network clock ran UTC-6, so every host stamp reads +1 h	
4	Was there a breach?	<code>issen info dc01.duckdb</code>	107 failed logons beside a service-install spike	
5	Initial vector	<code>issen timeline</code> <code>dc01.duckdb --event-type LogonSuccess --ip</code> <code>194.61.24.102</code>	RDP brute force , <code>Administrator</code> from <code>194.61.24.102</code> , success 03:21:48 (2 s after 107 fails)	
6.1	Malicious process	<code>issen memory "\$DC_MEM"</code> <code>--command ps</code>	coreupdater.exe (3644)	
6.2	Delivery IP	<code>issen timeline</code> <code>dc01.duckdb --ip</code> <code>194.61.24.102 --group-</code> <code>by event_type</code>	<code>194.61.24.102</code>	
6.3	C2	<code>issen memory "\$DC_MEM"</code> <code>--command netstat</code>	203.78.103.109:443 ESTABLISHED, owned by <code>coreupdater</code>	
6.4-6.5	On disk / first seen	<code>issen timeline</code> <code>dc01.duckdb --path</code> <code>'*coreupdater*' --first</code>	<code>C:\Windows\System32\coreupdater.exe</code> , first seen 03:24:06	

6.6	Moved?	<pre> issen timeline dc01.duckdb --path '*coreupdater*' -- event-type FileRename </pre>	USN rename trail	
6.7-	Capabilities /	<pre> issen memory "\$DC_MEM" </pre>	RWX private region + mZ header in spoolsv	
6.8	obtainable	<pre> --command scan </pre>	(3724) — injection MEASURED ; the <i>family</i> label (Meterpreter) is a YARA/VT call	○
6.9	Persistence	<pre> issen timeline dc01.duckdb --event- type ServiceInstall -- service coreupdater </pre>	7045 service install 03:27:49 (+ Run key)	
7	Malicious IPs (+ known infra?)	<pre> issen timeline dc01.duckdb --event- type LogonSuccess -- group-by ip </pre>	IPs + roles measured; "known infra" is OSINT (retracted link)	 ○
8	Lateral movement	<pre> issen timeline desktop.duckdb --event- type LogonSuccess --ip 10.42.85.10 </pre>	to the Desktop, 03:36:24	
8.3	Data stolen / when	<pre> issen timeline desktop.duckdb --path '*loot.zip*' </pre>	secret.zip / loot.zip staged-and-deleted ~02:30-02:34	
9	Network layout	<pre> issen timeline dc01.duckdb --tag system-info </pre>	adapter config from the SYSTEM hive: CITADEL-DC01 / C137.local , 10.42.85.10 /24, gw 10.42.85.100 + per-host logon IPs	
10	Architecture changes	analyst judgement	advisory layer — issen supplies the evidence	○
11	Szechuan sauce / time	<pre> issen timeline dc01.duckdb --path '*Szechuan*' </pre>	MFT access trail measured (Szechuan Sauce.txt accessed ~02:32), exfil inside the secret.zip window ~02:30-02:34; the wire transfer itself is PCAP	 ○
12	Other sensitive files	<pre> issen timeline dc01.duckdb --path '*beth*' </pre>	full trail measured; the Beth_Secret.txt timestamp is auto-flagged Medium (no analyst step)	
13	Last contact	<pre> issen timeline ... -- </pre>	measured: latest Logoff 04:52:11Z + C2	

		event-type Logoff -- last + issen memory ... --command netstat	203.78.103.109:443 still ESTABLISHED at capture = adversary live when imaged; issen session tightens the exact envelope	
B4-	Who logged	issen timeline	distinct logon users per host	✓
B5	on (DC/Desktop)	dc01.duckdb --event- type LogonSuccess -- distinct user		
B6	Domain passwords	issen memory "\$DC_MEM" --command creds	NTLM hash material recovered (validated hashdump) + SAM/LSA-secret inventory; turning a hash into plaintext is the offline-lab crack	✓/○
B7	Recover Beth's original	issen timeline dc01.duckdb --source RecycleBin	content recovered from \$R: SECRET_beth.txt → "Earth beth is the real beth." (size-matched)	✓
B8	Which file timestomped	issen timeline dc01.duckdb --flagged (scan stage)	Beth_Secret.txt auto-flagged Medium — whole- second \$SI back-date vs precise \$FN (stomped to match PortalGunPlans.txt)	✓
B1- B3	Controls / architecture	analyst judgement	map measured findings -> CIS/SANS	○

~24 of 27 questions fall out of a handful of commands off the one-command pipeline; the genuine ○ remainder is narrow — **known-infra attribution (OSINT), the wire-transfer (PCAP), plaintext password cracking, and the advisory layer.** Every measured answer above is taken from the real CitadelDC01 / Desktop images, validated against the union of the DFIR Madness official + bonus answer keys.

8 · Correlation — One Timeline, Many Sources

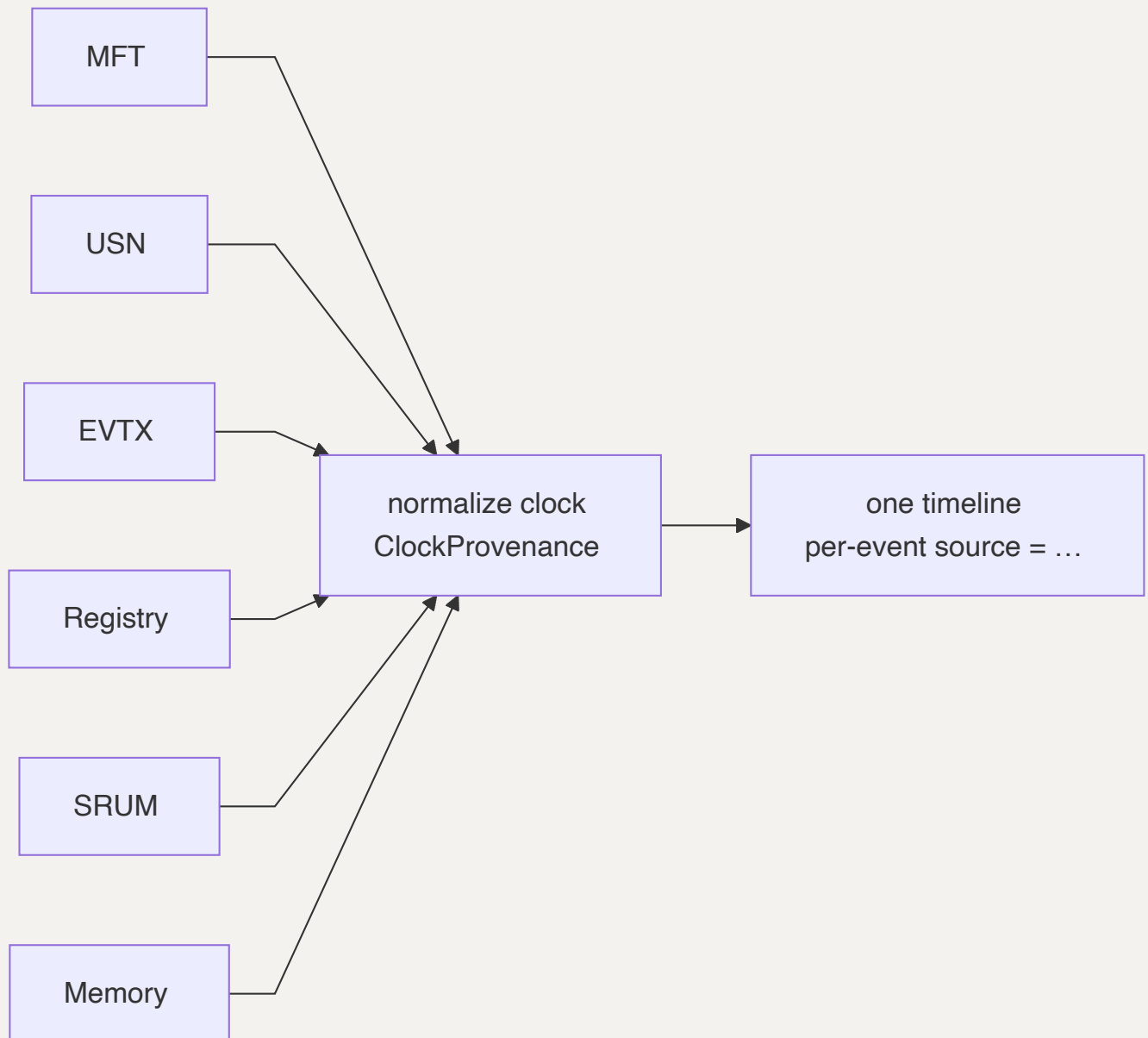
Each layer above produces events in **its own** address space. Correlation **merges them into one super-timeline** — with **per-event source attribution**.

- Disk \$MFT + USN + EVTX + Registry + SRUM + Memory → **one ordered narrative**.
- Each row carries **where it came from** (source = EVTX | MFT | USN | memory)

| SRUM).

- Clock skew normalized once (ClockProvenance), so every source lands on the **same wall clock**.

*The breach isn't proven by one artifact — it's proven by **five independent sources agreeing** on the same minute.*



The Climax — ATT&CK Narrative & The Report

The deliverable is not a tool dump. It's an **attack-chain narrative** a board can read:

T1110 brute force → T1021.001 RDP → T1543.003 service persistence → T1055 injection → T1071/T1573 C2 → T1070.006 timestamp → T1560/T1041 stage & exfil.

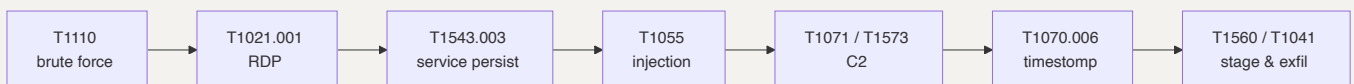
Built from **graded findings**, each tagged "*consistent with*" — never a verdict.

```

issen DC01-E01.zip citadelc01.mem -o case.duckdb # build it:
ingest + correlate + scan + memory, one pass
issen report case.duckdb --format text # the attack-
chain narrative, live in the terminal
issen report case.duckdb # the same, as
a board-ready HTML report

```

Write every sentence in one of three layers: ① observed fact (state it) → ② forensic inference ("consistent with", never "proves") → ③ legal conclusion (hand to the tribunal). Never let a layer-① finding wear layer-③ language. (See "Present It Like an Expert Witness.")

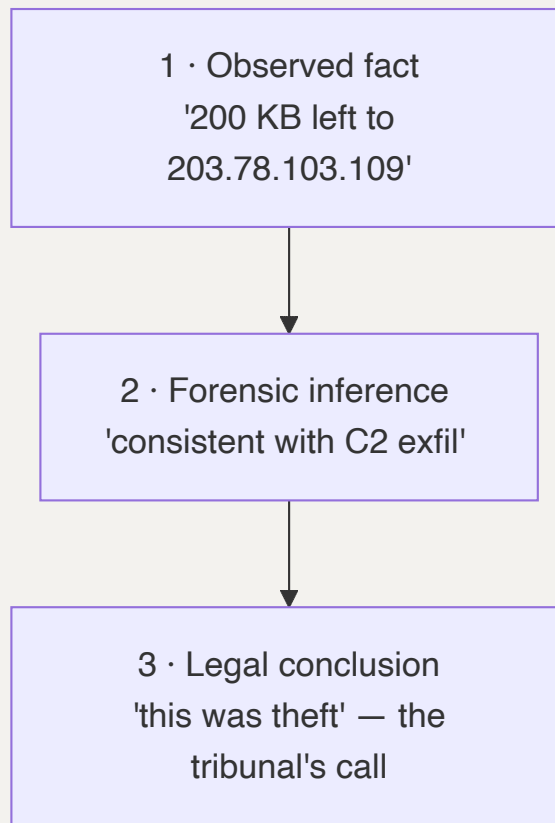


Present It Like an Expert Witness

The discipline that separates an analyst from a technician — **three epistemic layers**:

1. **Observed fact** — "200 KB left the host to 203.78.103.109 at 02:24." → state it.
2. **Forensic inference** — "consistent with C2 exfiltration." → "consistent with", never "proves".
3. **Legal conclusion** — "this was theft." → **not yours**. "The tribunal may draw its own conclusions."

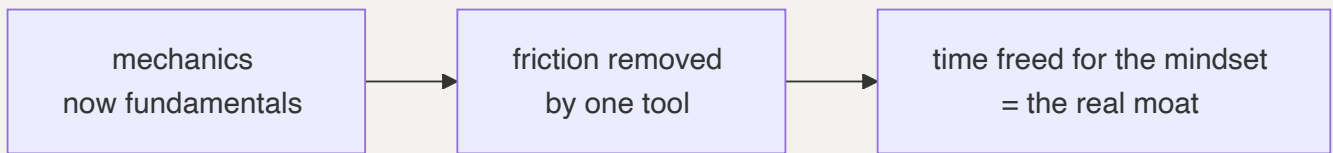
Tools find the bytes. **Judgment** decides what they mean — and **restraint** decides what you're entitled to say.



The Takeaway

- The **mechanics** — containers, partition tables, NTFS, EVTX, registry, SRUM, memory — are **fundamentals you now recognize anywhere**.
- The **tooling** — one Rust-native fleet — collapses four ecosystems into one address space, so the friction stops stealing your thinking time.
- The **moat** — and your career — is the **mindset**: read the meaning, build the narrative, present with honesty.

Now let's open the image. → *Module 1: crack the container.*



More From the Fleet · sqlite4n6

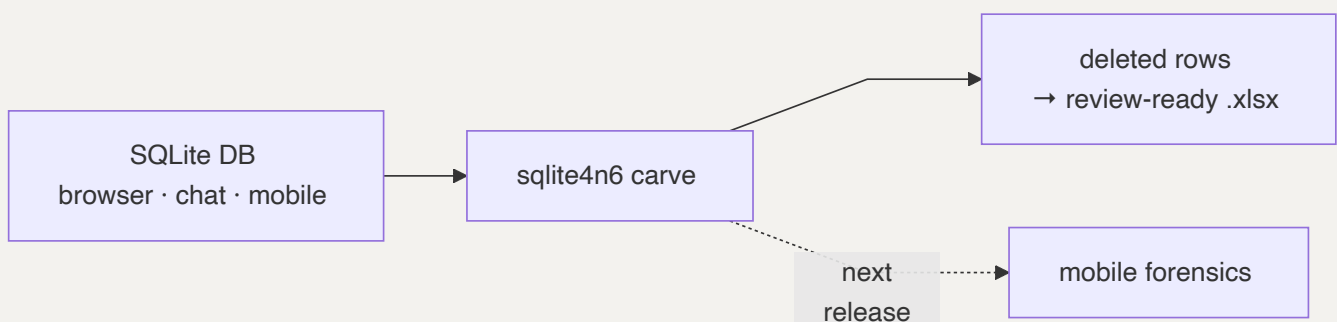
The deleted rows are the evidence — and `sqlite3` can't see them.

`sqlite-forensic` carves deleted records out of any SQLite database — browser history, chat apps, mobile artifacts — into a **review-ready spreadsheet**: live, prior-changed, and deleted versions recovered from the uncheckpointed WAL, the rollback journal, and free space. Opens evidence **read-only**, never re-surfaces a live row as "deleted."

```
sqlite4n6 carve History.db      # → History.recovered.xlsx
```

Coming next: mobile forensics. Most phone evidence is SQLite — the next release turns `sqlite4n6` into first-class iOS / Android app-database recovery.

The byte-layout diagrams in this deck come from its forensic paper. ·
securityronin.github.io/sqlite-forensic



More From the Fleet · 4n6mount

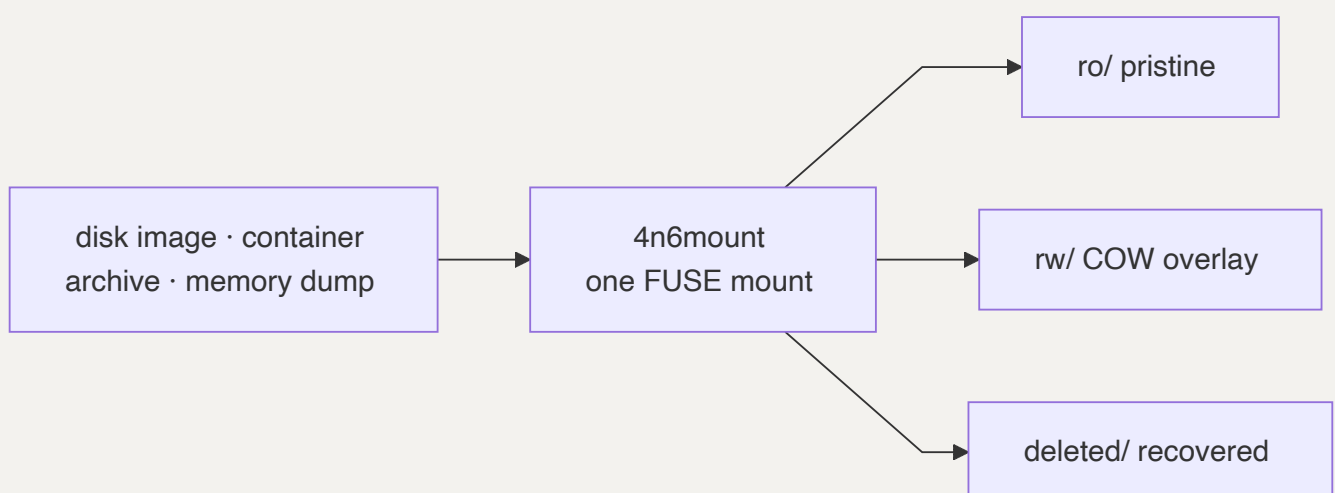
Mount the image. Browse evidence like files. Write without touching the original.

— one FUSE mount, every format.

- **Auto-detects the source** — ext4 / NTFS / exFAT / HFS+ / APFS / ISO9660 filesystems, EWF & VMDK containers, zip / 7z / tar archives, even **memory dumps** — all through one command.
- **Read-only evidence beside a copy-on-write overlay** — run grep, save notes, pipe output against `rw/`; every write lands in a sidecar and the image stays byte-for-byte pristine.
- **deleted/ recovery, metadata/timeline.jsonl, hash-based filtering** — an analysis surface, not just a raw mount, so any tool that reads files now reads evidence.

```
4n6mount image.dd /mnt/evidence # ro/ rw/ deleted/ metadata/
```

The bridge under the fleet: scriptable, CI-friendly, chain-of-custody safe — pipe deleted files straight into `issen`, `br4n6`, or `blazehash`. · securityronin.github.io/4n6mount



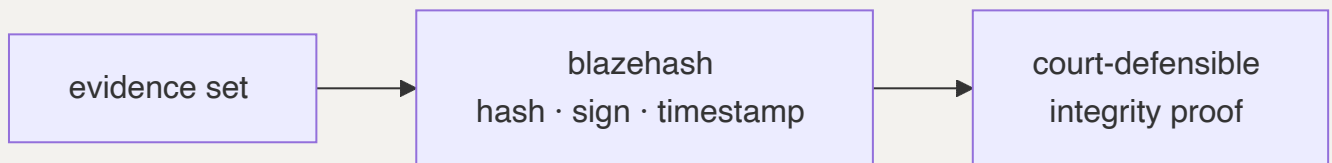
More From the Fleet · blazehash

Hash. Sign. Timestamp. Prove. Seal. — integrity hashing for workflows that end in a courtroom.

- **BLAKE3 at 1,640 MB/s**, 25 hash algorithms — hash a whole evidence set fast.
- **Ed25519 + post-quantum signing** and **Bitcoin-anchored timestamps** — prove a file existed, unaltered, at a given time.
- **hashdeep-compatible** — every flag and output format you already use.

```
brew install securityronin/tap/blazehash
```

*Built for **legal proceedings**: a hash set you can put in front of a tribunal — verifiable, signed, and timestamped, not merely computed. · securityronin.github.io/blazehash*



Thank You

Build the narrative. Present with honesty. That's the moat.

Author: Albert Hui · linktr.ee/4n6h4x0r

IR Peer Review: Eliza Wan · linkedin.com/in/eliwan

QA: Josiah Wu · jwu29-blog.com

Backup / Reference

- Case: **DFIR Madness Case 001** — <https://dfirmadness.com/the-stolen-szechuan-sauce/>
- Answer key: <https://dfirmadness.com/answers-to-szechuan-case-001/>
- Clock: victim VMs **UTC-7**; key narrates **UTC-6** → host artifacts read **+1h**.